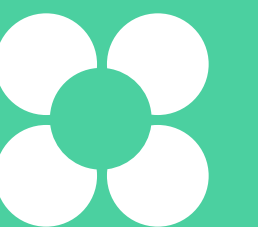


Работа с локальным репозиторием

Алёна Батицкая
Фронтенд-разработчик, фрилансер



Алёна Батицкая

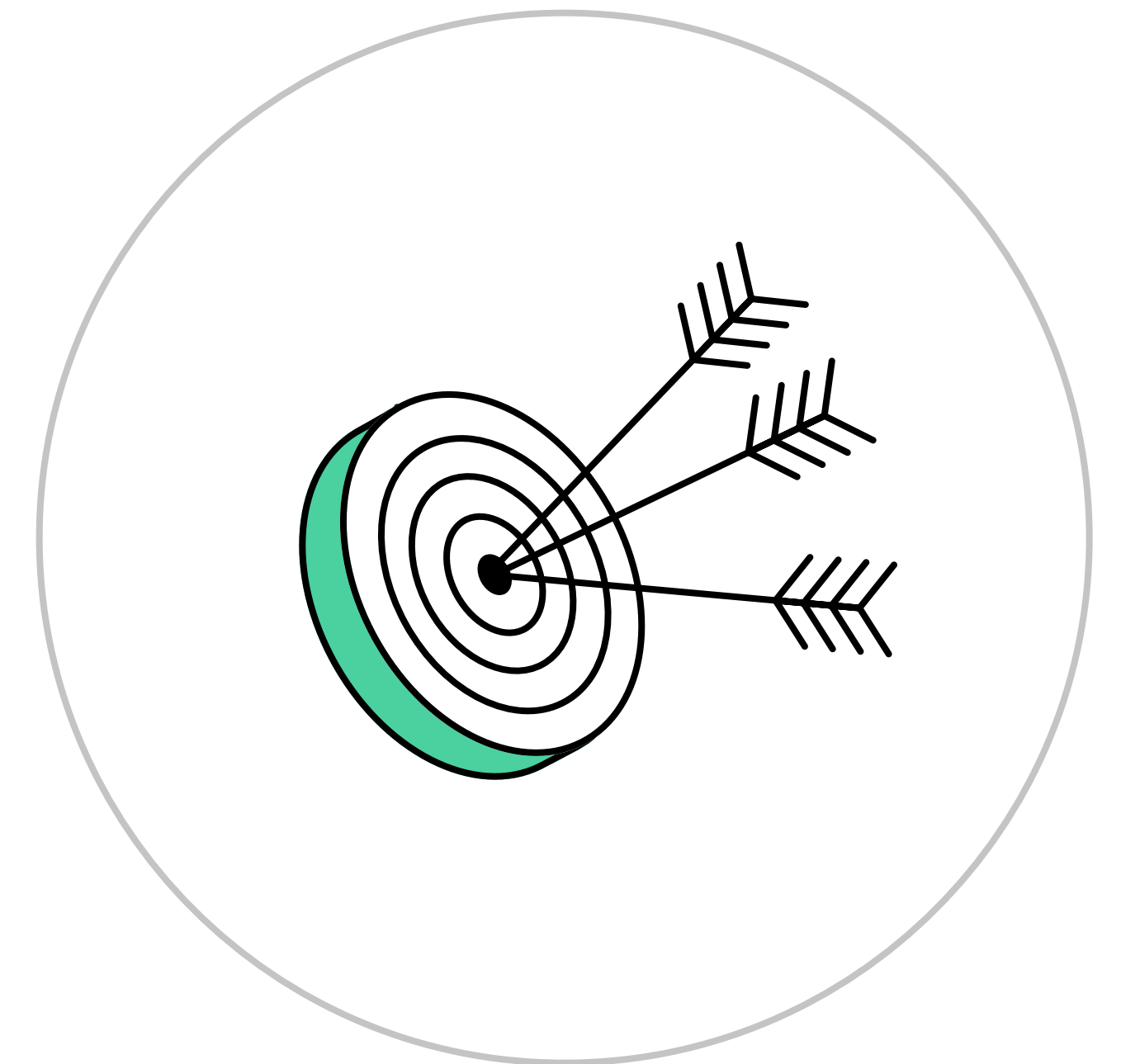
О спикере:

- Фронтенд-разработчик на фрилансе
- Автор и руководитель курсов в Нетологии
- Google Developer Expert for web
- Редактор проекта «Дока»
- Спикер, автор и переводчик статей по программированию



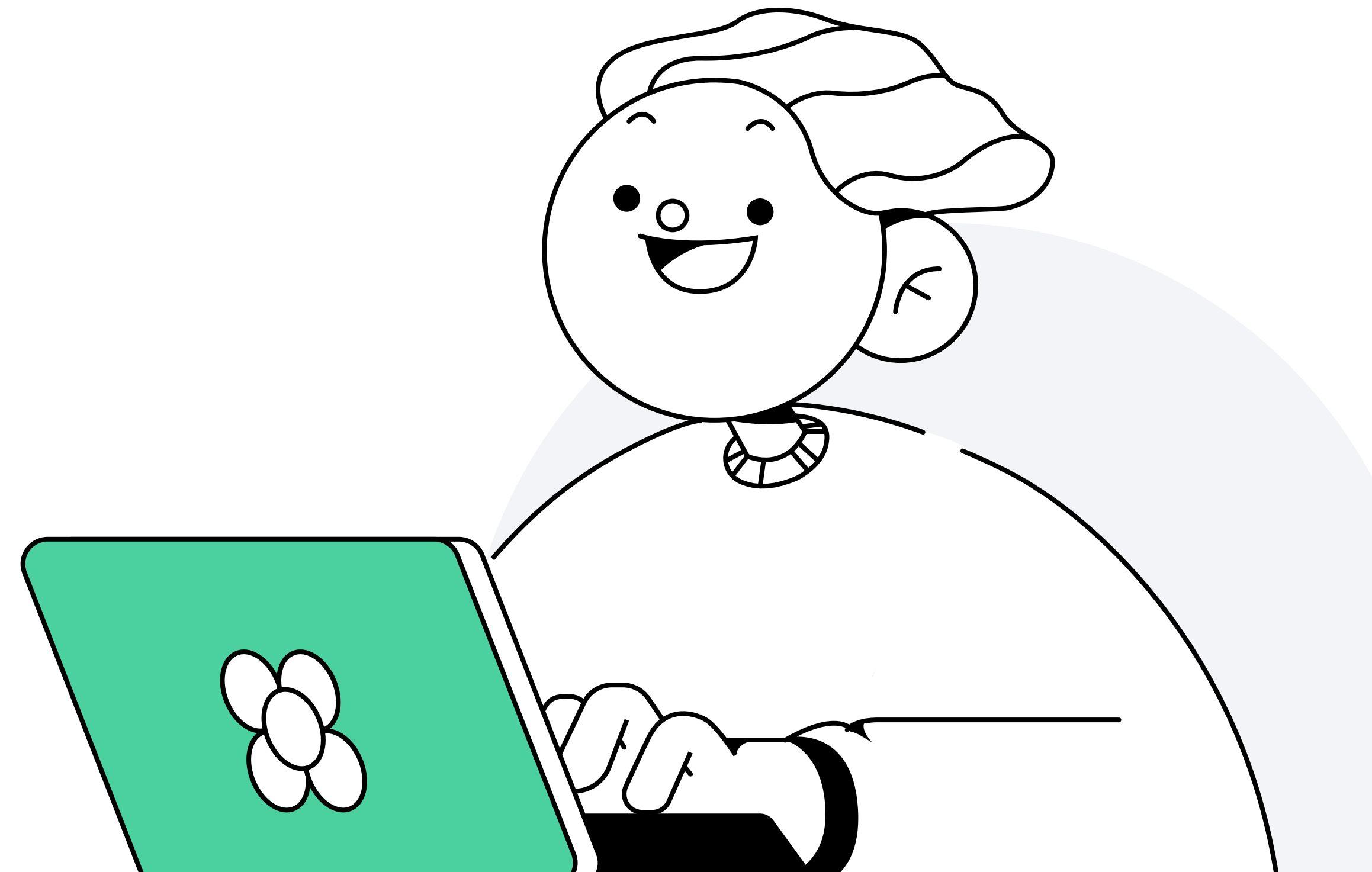
Цели занятия

- Создать первый проект в Git на локальном компьютере
- Научиться фиксировать, отслеживать и править существующий коммит в проекте
- Научиться перемещаться по истории проекта



План занятия

- 1 Создание локального репозитория
- 2 Работа с историей
- 3 Исправление локального коммита



Создание локального репозитория

Скринкаст

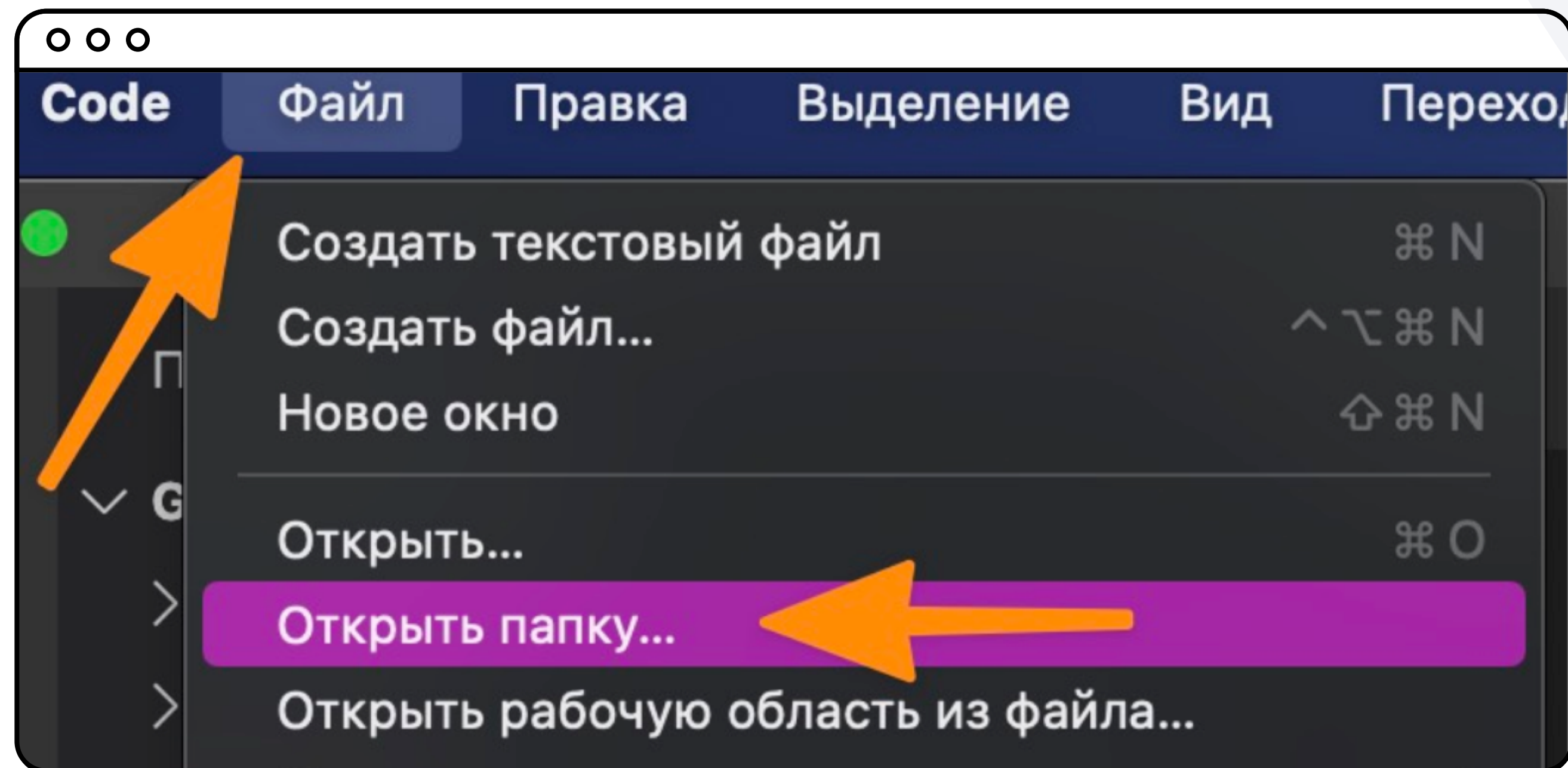
Первый проект test для Git. Создание файла README.md через Visual Studio Code



Создание проекта для Git

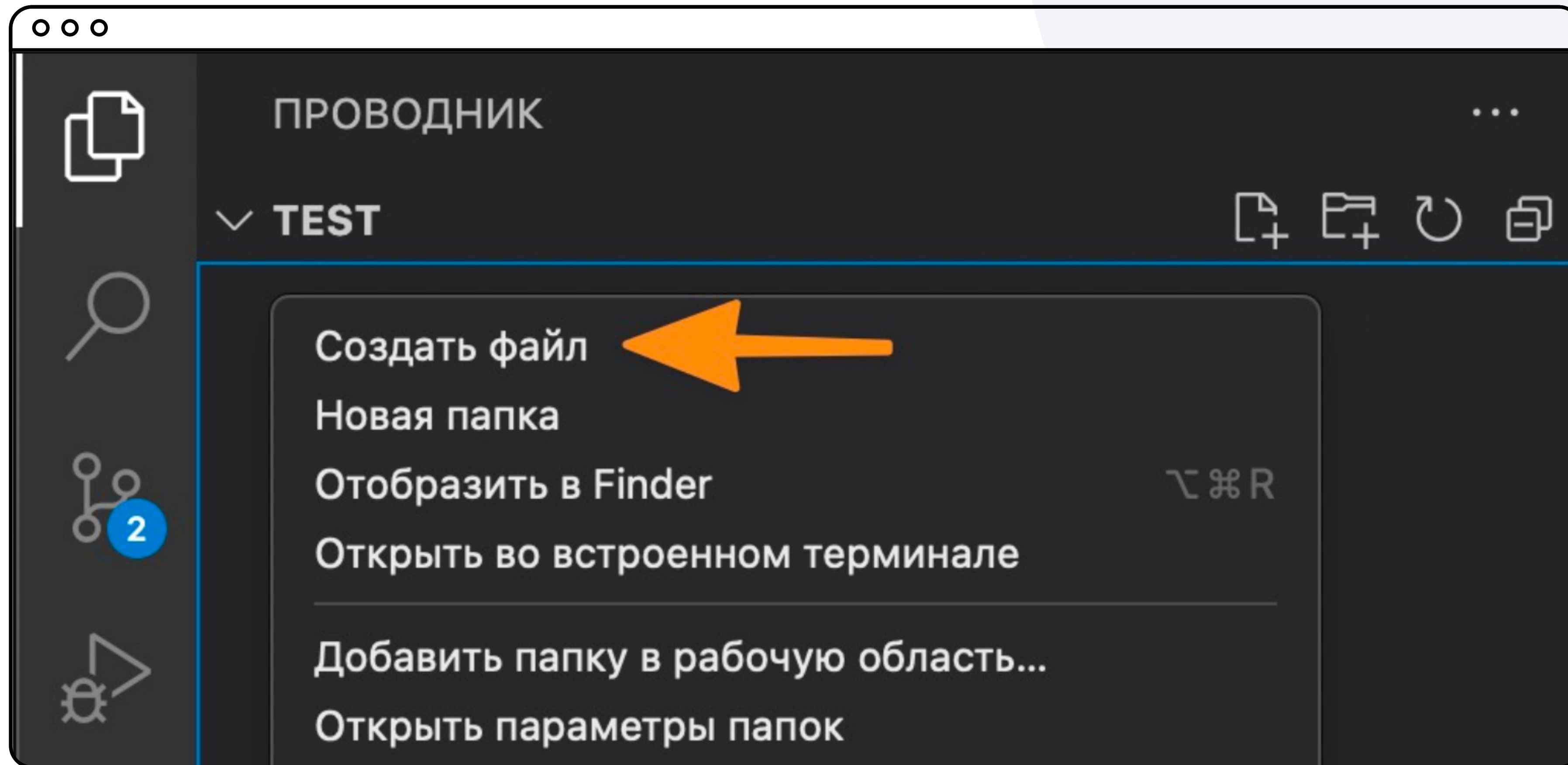
- 1 Создайте на рабочем столе папку с названием **test**
- 2 Откройте **Visual Studio Code**. В нём откройте папку **test** через «Открыть...» (Open...) / «Открыть папку...» (Open folder...)

Или в верхнем меню выберите Файл (File) → Открыть папку... (Open folder...) и выберите в проводнике папку test



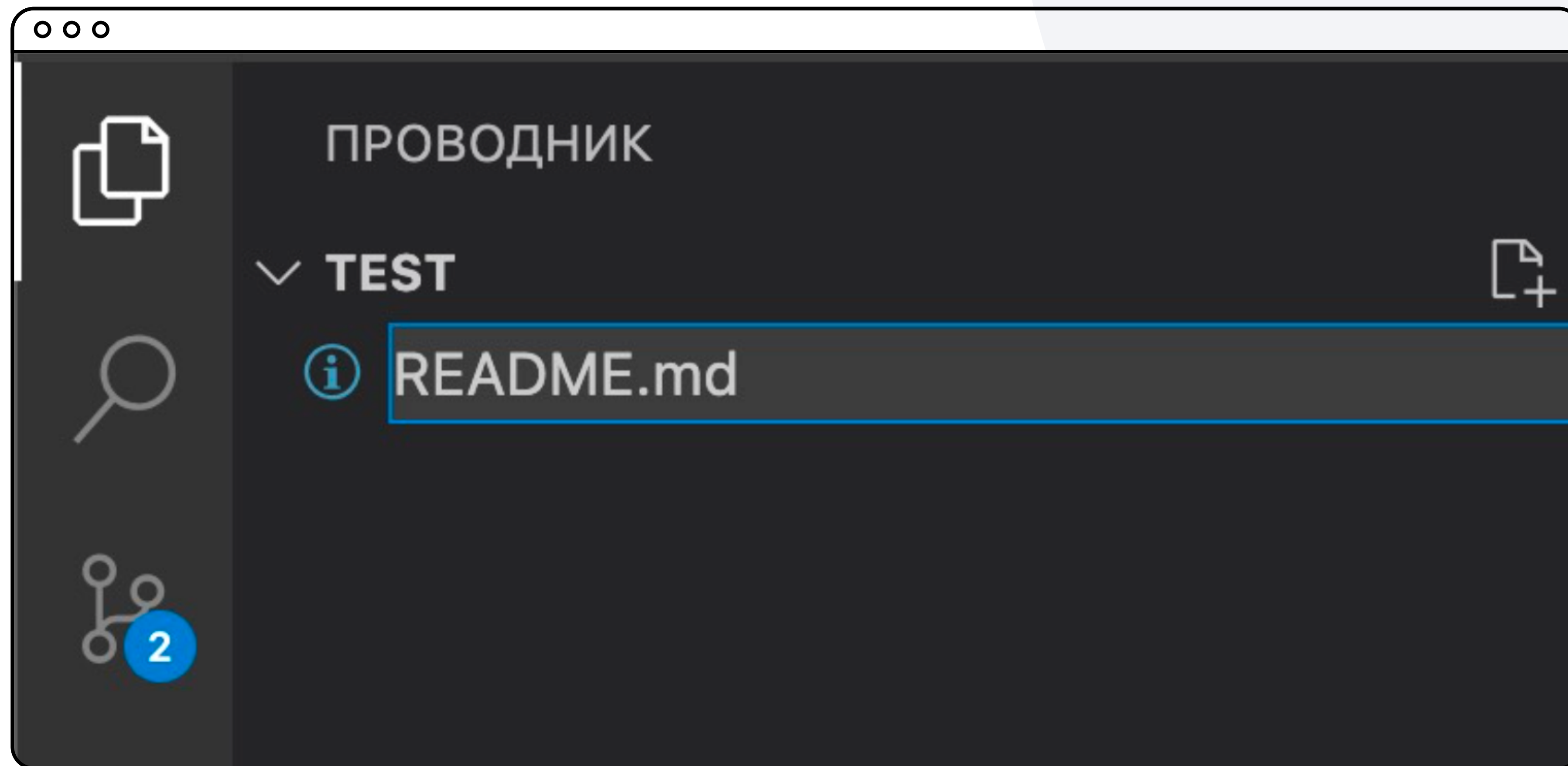
Создание проекта для Git

- 3 Кликните правой кнопкой мыши в левой колонке под названием **test** и выберите **Создать файл (New file)**



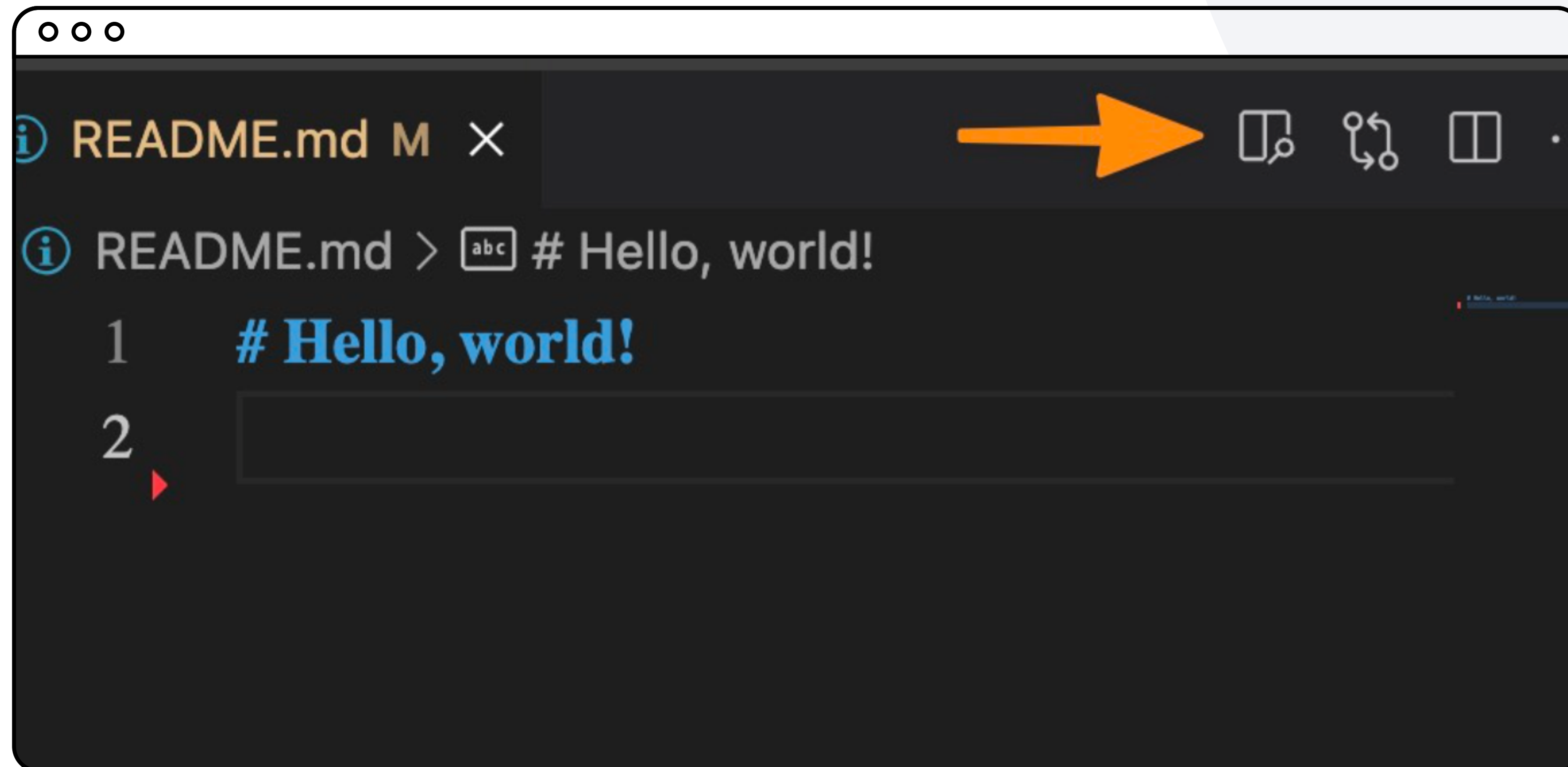
Создание проекта для Git

- 4 В появившемся поле введите название файла **README.md**



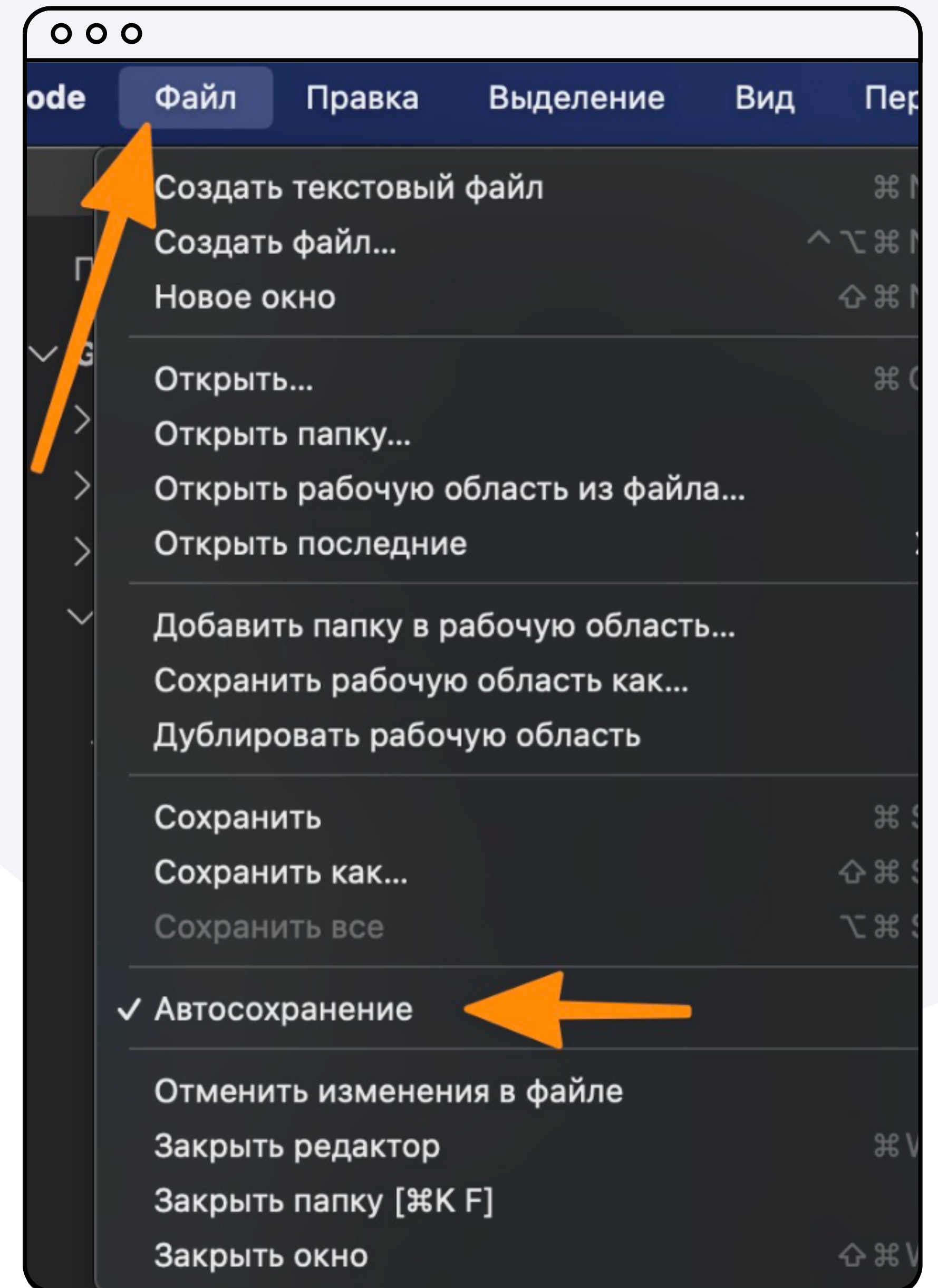
Создание проекта для Git

5. Внутри файла напишите строку, используя Markdown **# Hello, world!**
6. Убедитесь, что всё верно написали и заодно протестируйте плагин предпросмотра Markdown



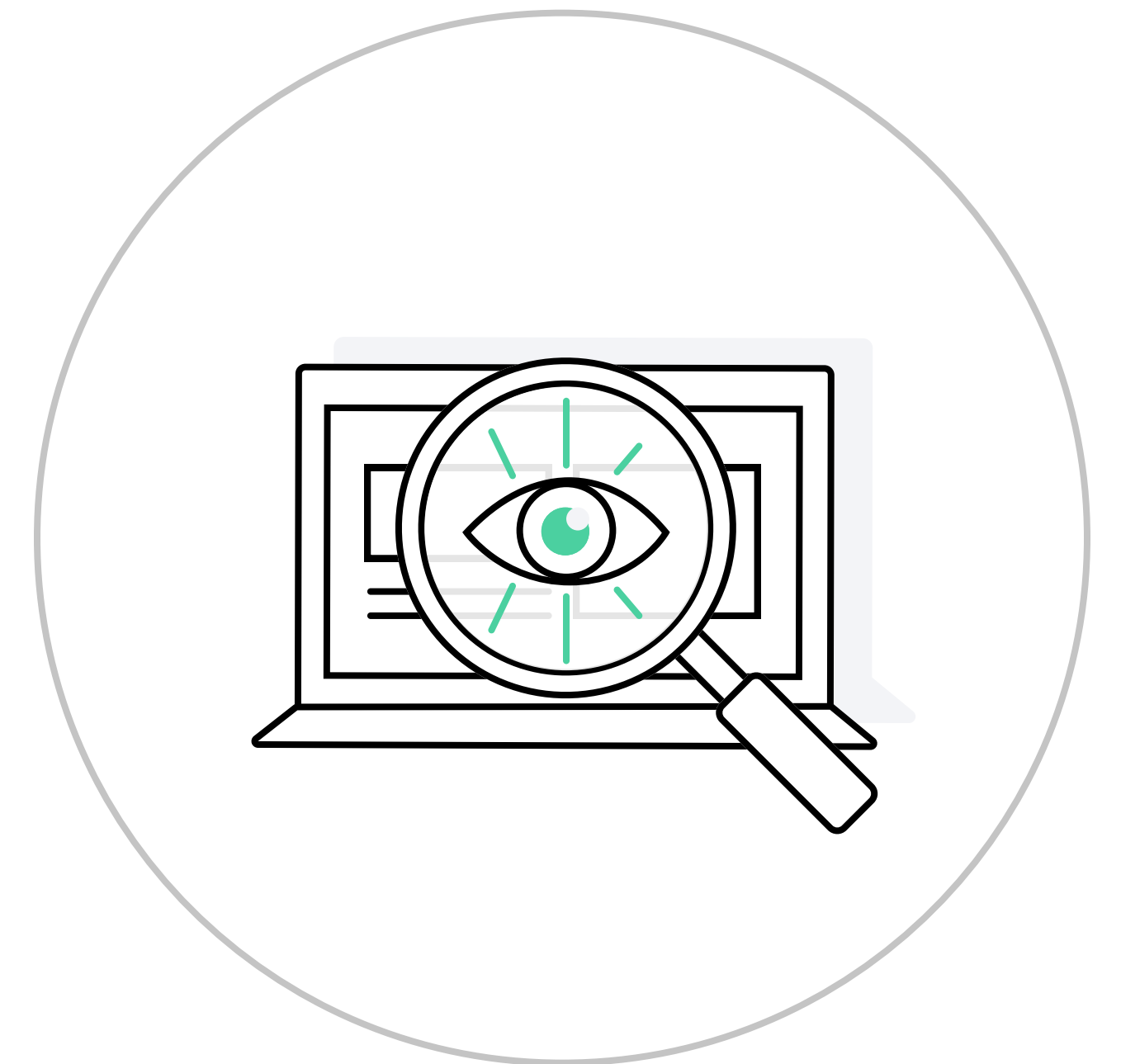
Создание проекта для Git

- 7 Если рядом с именем файла на вкладке стоит белая точка — это сигнал, что внесённые в файл изменения не сохранены
- 8 Обязательно сохраните файл. Для этого нажмите **Ctrl + S** или в верхнем меню выберите **Файл (File) → Сохранить (Save)**
- 9 Для удобства работы настройте автосохранение. В верхней строке нажмите на **Файл (File)** и поставьте галочку рядом с пунктом **Автосохранение (Auto Save)**



Repository (репозиторий)

Папка, за которой следит Git



Виды репозиториев



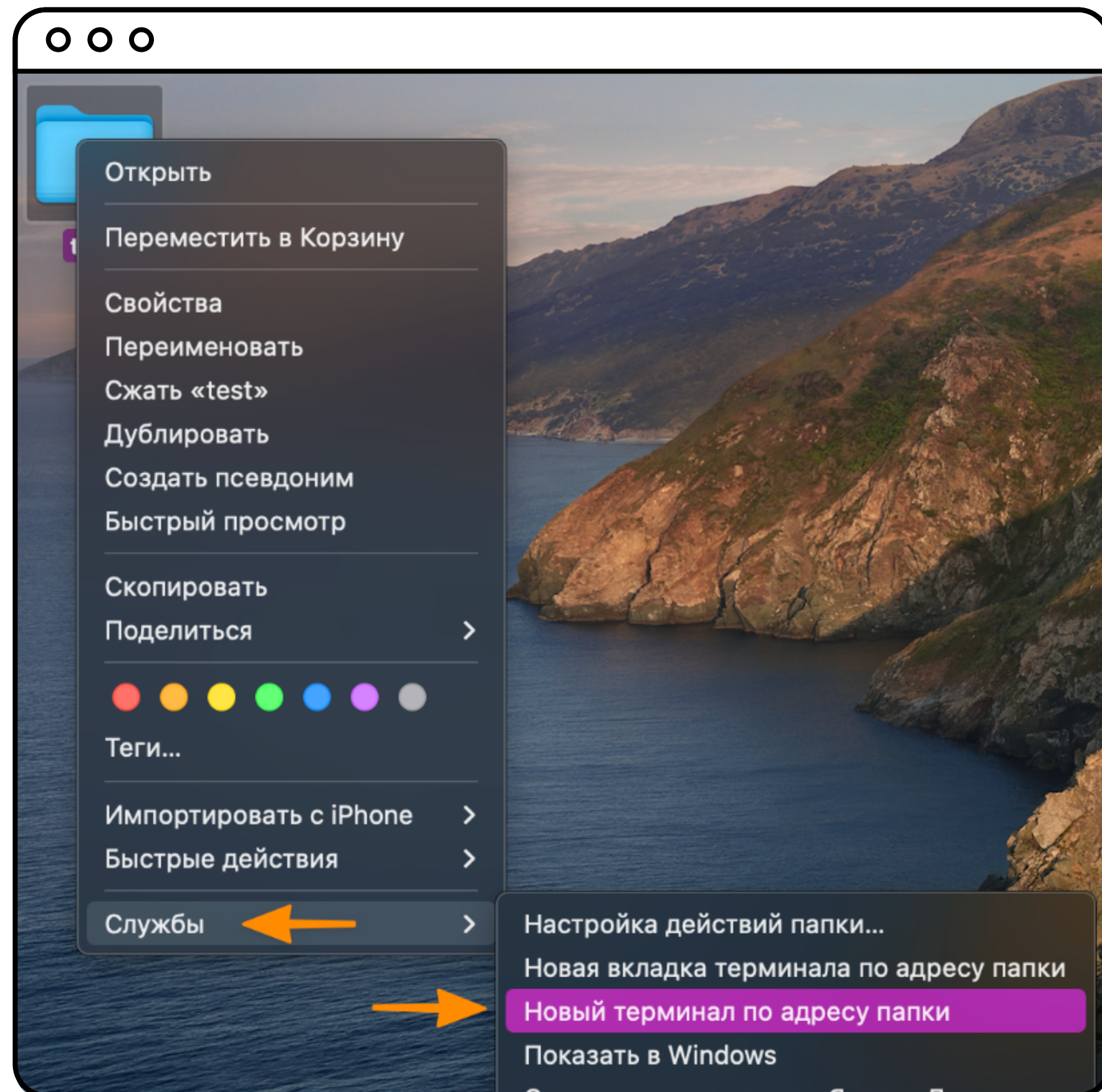
Скринкаст

Настройка связи между папкой test и Git



Связывание папки и файла с Git

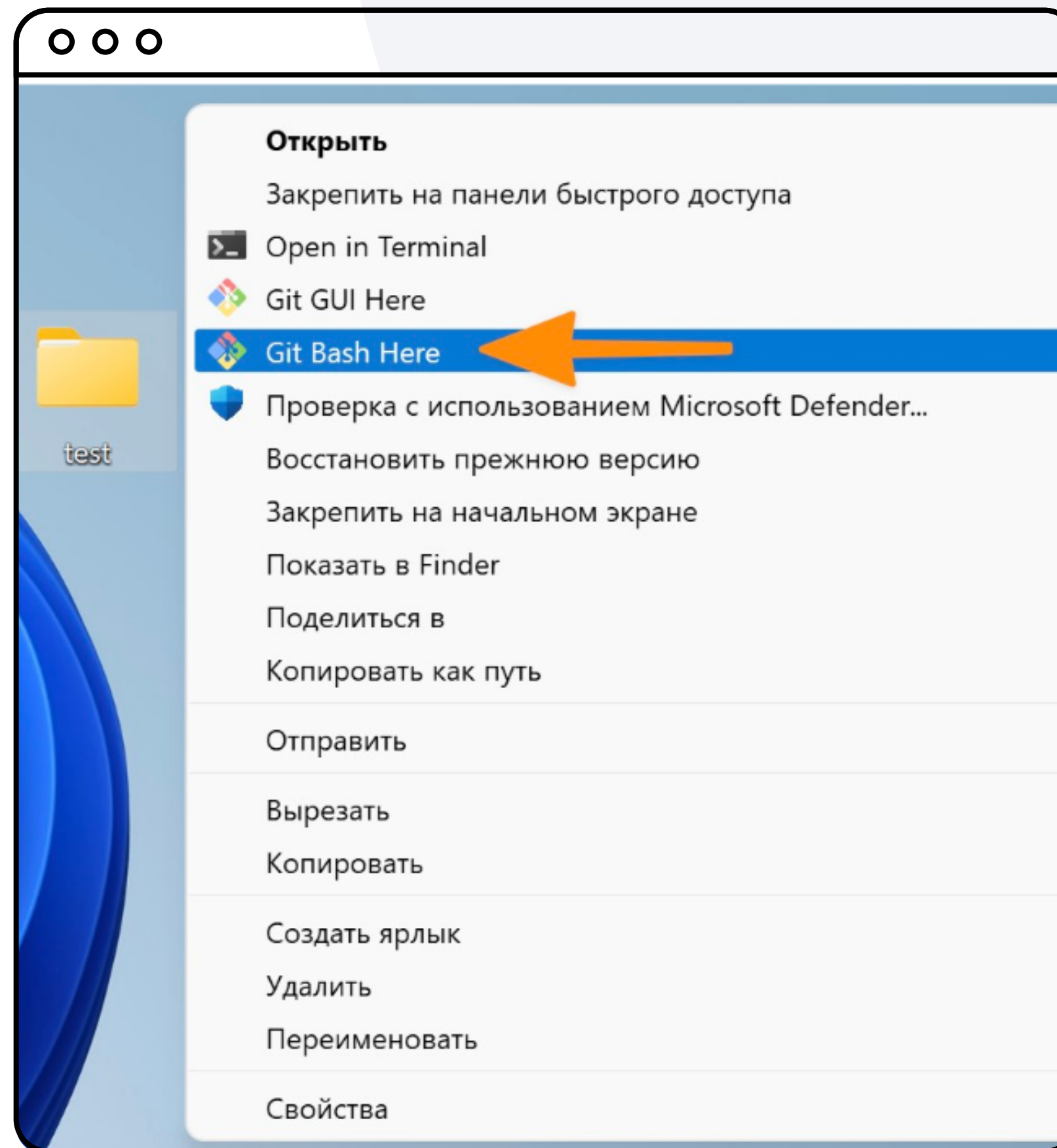
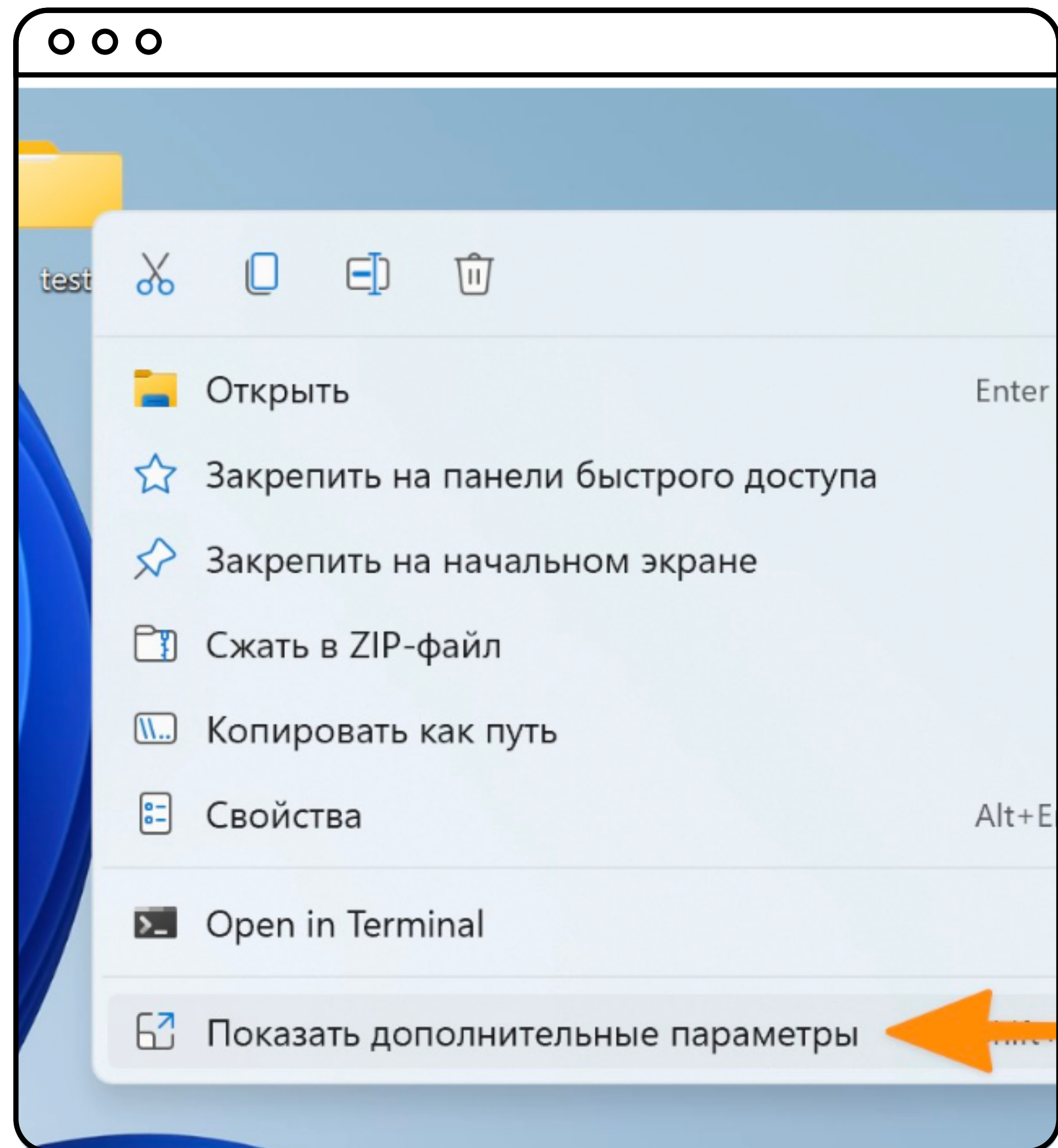
- 1 Для **macOS** на рабочем столе найдите папку **test**, нажмите на неё правой клавишей мыши, выберите **Службы** → **Новый терминал по адресу папки**



Скриншот macOS

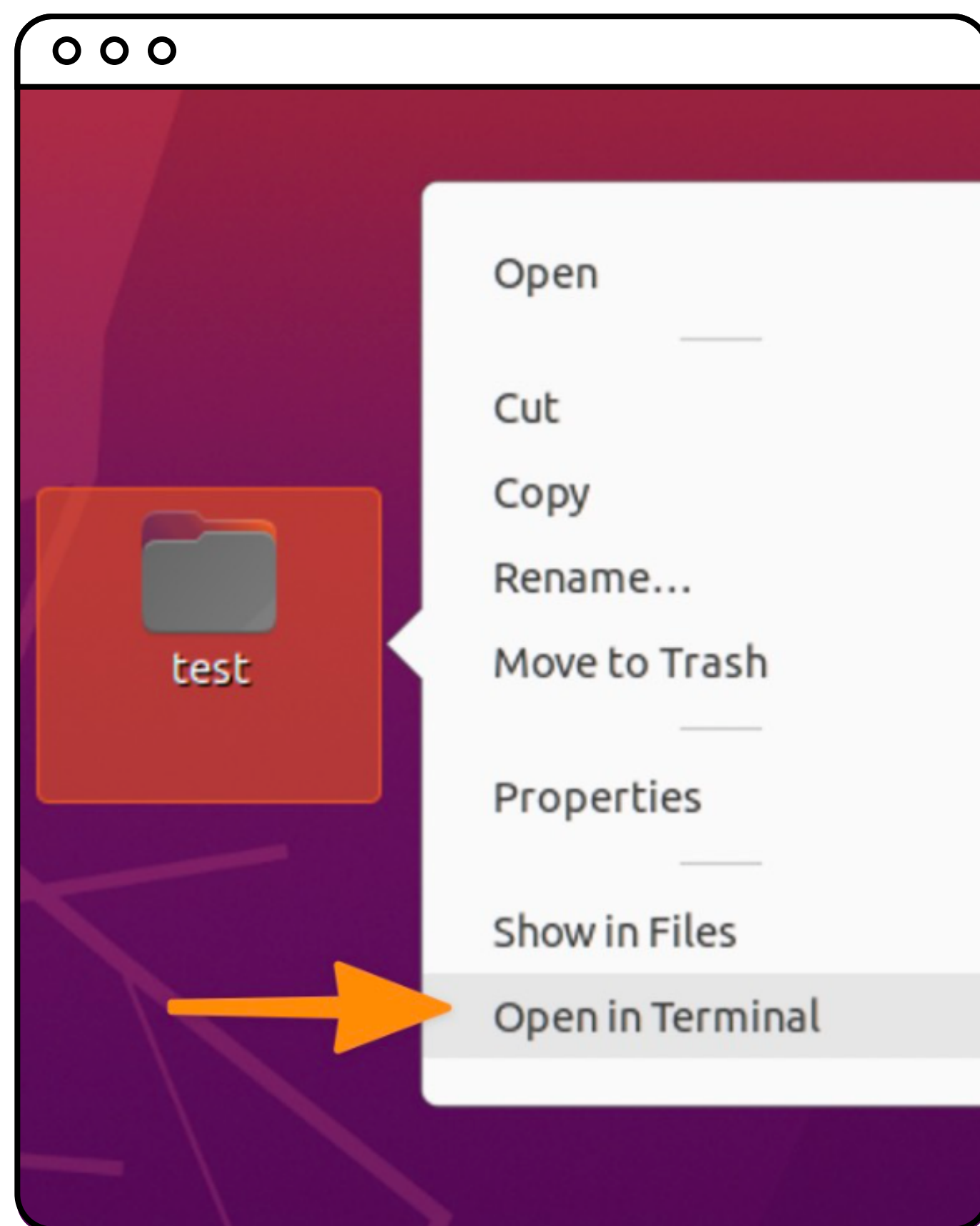
Связывание папки и файла с Git

- 1 Для **Windows 11** на рабочем столе найдите папку **test**, нажмите на неё правой клавишей мыши, выберите **Показать дополнительные параметры** → **GIT Bash here**.



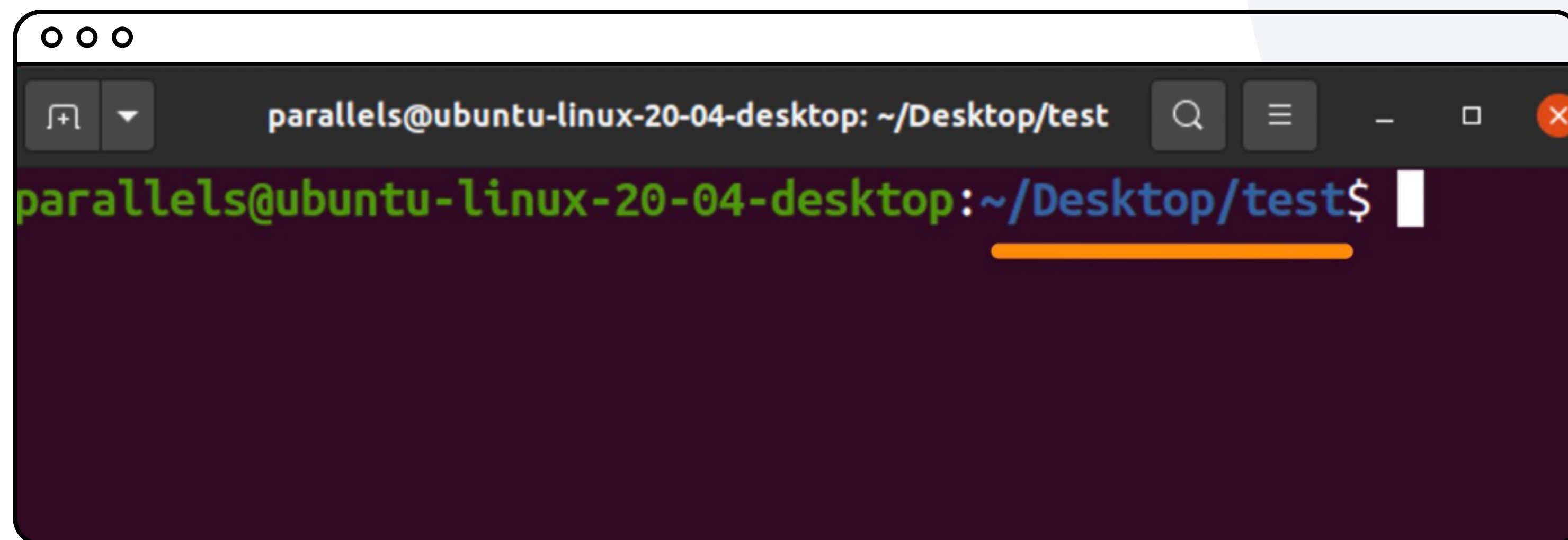
Связывание папки и файла с Git

- 1 Для **Linux** на рабочем столе найдите папку **test**, нажмите на неё правой клавишей мыши и выберите **Открыть в Терминале (Open in Terminal)**. Откроется окно терминала. В начале строки в терминале виден путь до папки, в которой вы сейчас находитесь



Связывание папки и файла с Git

Для всех операционных систем. В начале строки в терминале виден путь до папки, в которой вы сейчас находитесь

A screenshot of a Linux terminal window. The window has a title bar with three small circles on the left and a search bar, menu icon, and window controls on the right. The title bar text is "parallels@ubuntu-linux-20-04-desktop: ~/Desktop/test". The terminal itself has a dark purple background. The prompt "parallels@ubuntu-linux-20-04-desktop:~/Desktop/test\$" is displayed in green and blue text. The path "~/Desktop/test" is underlined with an orange line. A white cursor is at the end of the prompt.

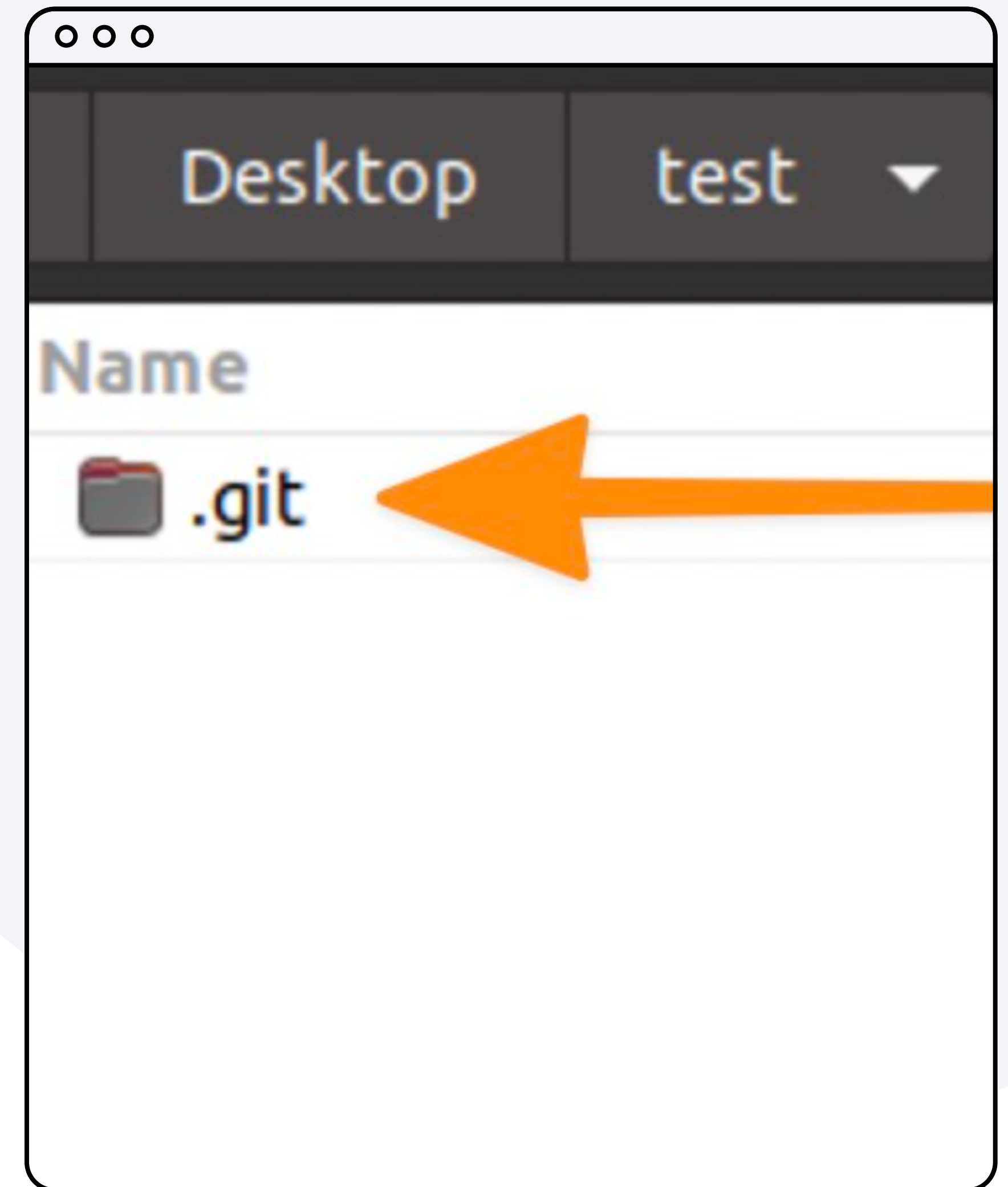
```
parallels@ubuntu-linux-20-04-desktop: ~/Desktop/test
parallels@ubuntu-linux-20-04-desktop:~/Desktop/test$
```


Связывание папки и файла с Git

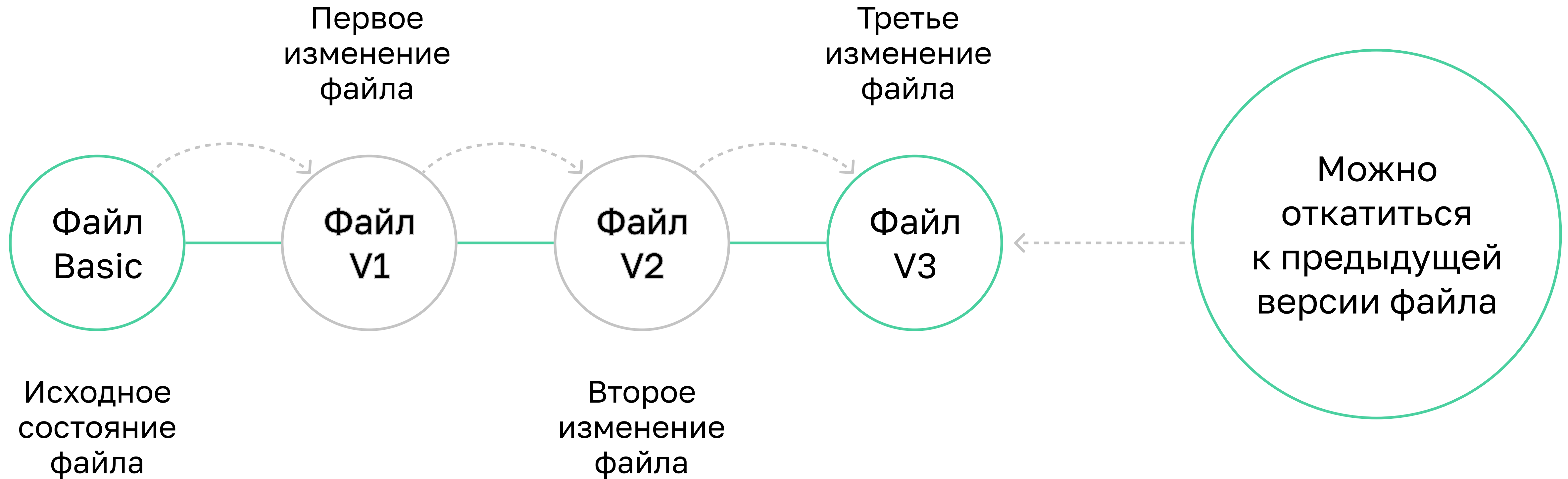
- 7 Чтобы прицепить Git к папке — создать локальный репозиторий — нужно выполнить команду **git init**. После выполнения команды в терминале появится строка:

Initialized empty Git repository in .../Desktop/test/.git/

- 8 Вы создали локальный репозиторий. Чтобы проверить это, откройте папку **test** в проводнике на компьютере и найдите там **папку .git**



Принцип работы Git



Состояния файлов в Git



Скринкаст

Настройка отслеживания файла README.md
и создание его снимка



Работа с файлами

- 1 Чтобы проверить текущее состояние репозитория, введите в терминале команду **git status**.
В ответ Git выдаст следующее сообщение, где файл **README.md** выделен красным

```
ooo
mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git status
on branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    README.md

nothing added to commit but untracked files present (use "git add" to track)

mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$
```


Работа с файлами

- ② Чтобы добавить файл README.md в отслеживаемые, выполните в терминале команду:
git add README.md, (где git add — это команда, а README.md — имя файла)
- ③ Если всё введено верно, то компьютер молча выполнит команду

Работа с файлами

- ④ Чтобы проверить, что файл README.md теперь отслеживается, выполните снова команду **git status**. При этом Git ответит, что может быть добавлен новый файл (new file). Имя файла теперь зелёное, значит Git теперь следит за файлом

```
o o o
$ git add README.md

mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git status
on branch main

No commits yet

changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   README.md

mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
```

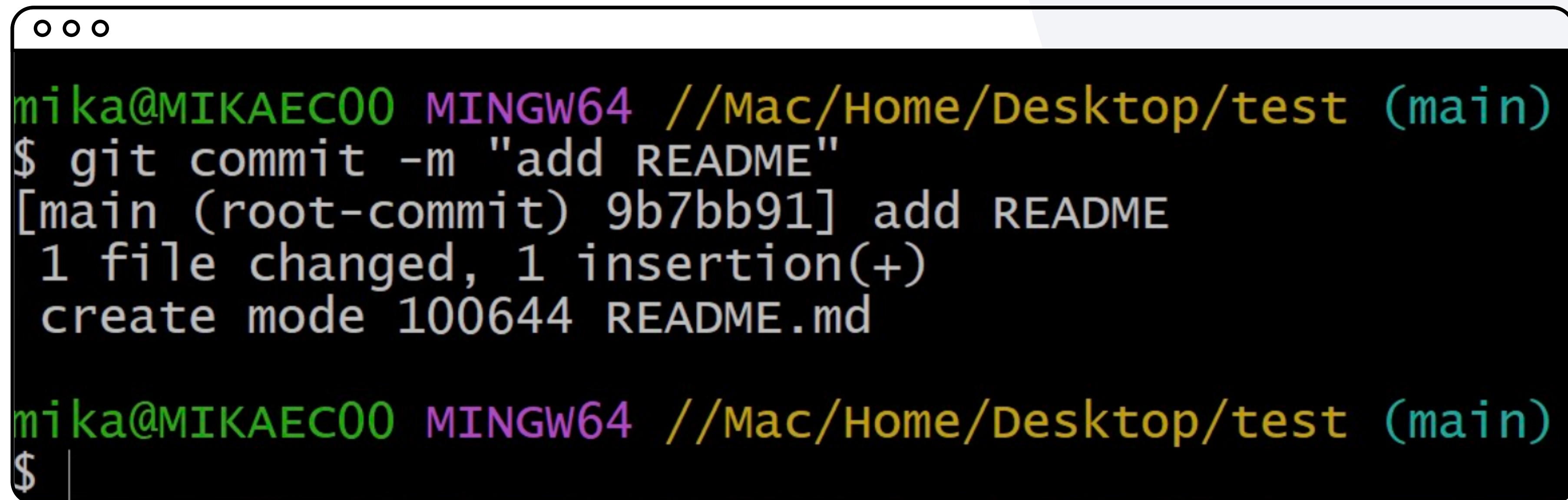
Commit (коммит)

Процедура, при которой делается снимок файла (папки)
в текущем состоянии



Работа с файлами

- 5 Выполните команду **git commit -m "add README"**. В кавычках принято писать о том, что изменилось в проекте с последнего снимка.

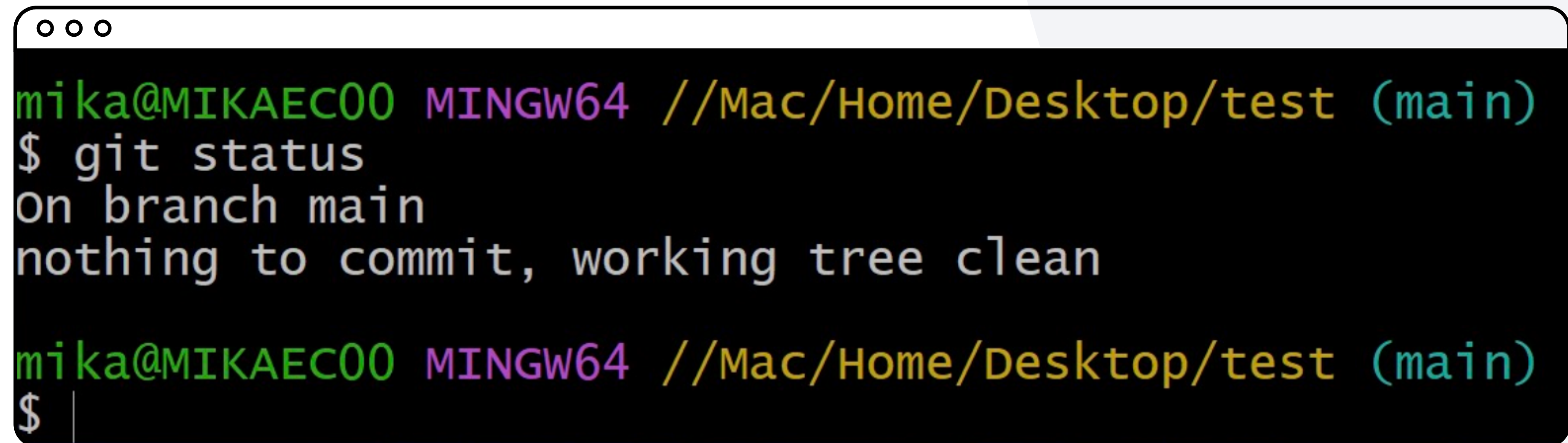
A terminal window with a dark background and light-colored text. The window title bar shows three small circles. The prompt is 'mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)'. The command '\$ git commit -m "add README"' is entered. The output shows the commit hash '9b7bb91', the message 'add README', and details about the file change: '1 file changed, 1 insertion(+)' and 'create mode 100644 README.md'. The prompt is repeated at the bottom, followed by a dollar sign and a vertical bar, indicating the command prompt is ready for input.

```
mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git commit -m "add README"
[main (root-commit) 9b7bb91] add README
1 file changed, 1 insertion(+)
create mode 100644 README.md

mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ |
```


Работа с файлами

- 6 Выполните команду **git status**, чтобы убедиться, что все файлы попали в снимок проекта

A terminal window with a dark background and light-colored text. The window has three small circles in the top-left corner. The text inside shows a user named 'mika' at a machine named 'MIKAE00' running a 'MINGW64' shell. The current directory is '//Mac/Home/Desktop/test' and the user is on the 'main' branch. The command '\$ git status' has been entered, and the output is 'on branch main' and 'nothing to commit, working tree clean'. The prompt '\$' is shown again at the bottom, followed by a vertical bar indicating the cursor position.

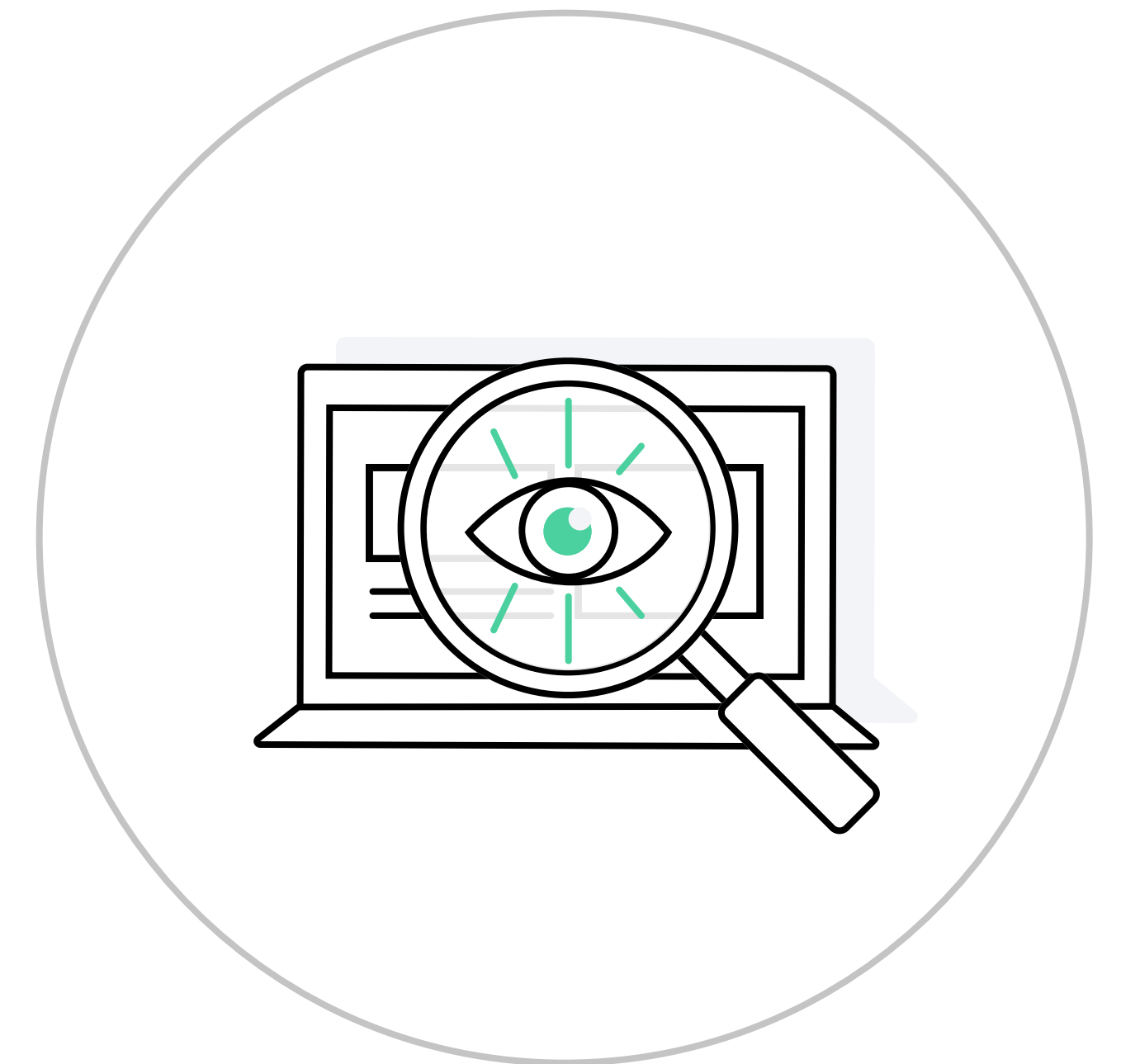
```
mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git status
on branch main
nothing to commit, working tree clean

mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ |
```


Последовательность действий

Чтобы создать локальный репозиторий:

- Создайте папку проекта на компьютере
- Откройте для папки терминал
- Выполните команду **git init**
- Проверьте, что появилась папка .git



Последовательность действий

Чтобы сделать снимок проекта (сформировать коммит):

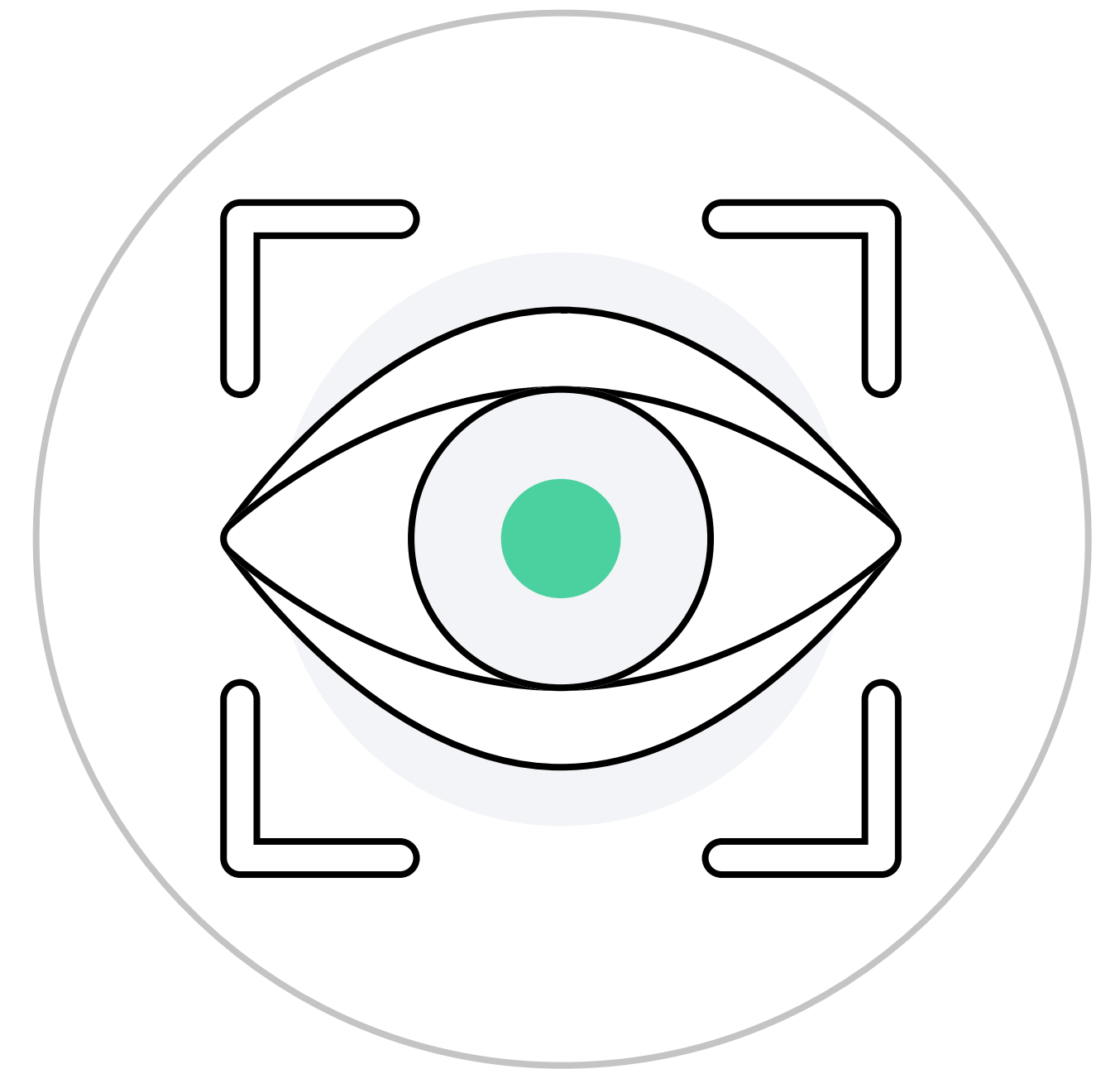
- Добавьте файл или файлы в отслеживаемые командой **git add имя файла**
- Создайте коммит и подпишите его командой **git commit -m "Подпись коммита"**



Последовательность действий

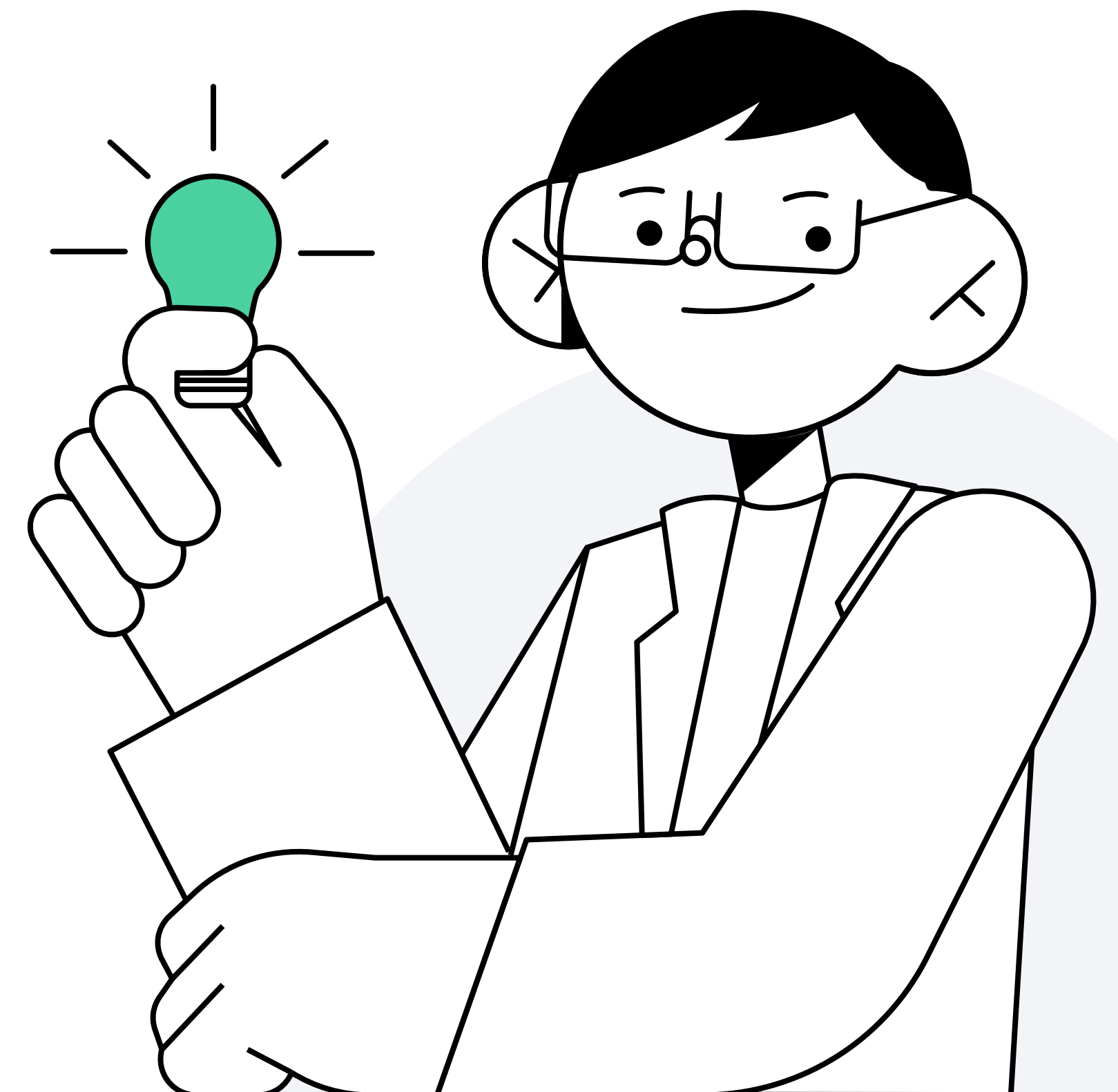
Чтобы проверить состояние файла

- На любом из этапов вы можете выполнить команду **git status**



Итоги

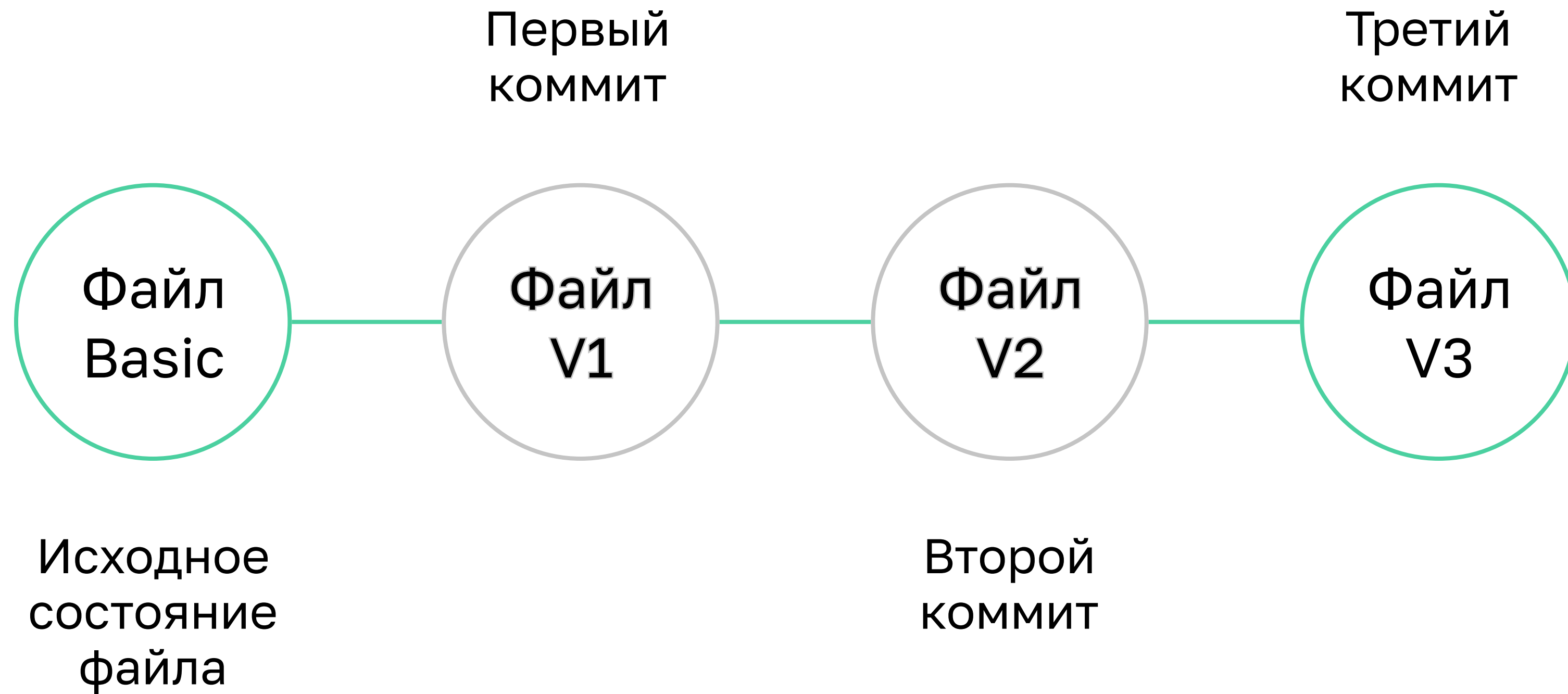
- ① Научились создавать локальный репозиторий
- ② Научились добавлять файлы в отслеживаемые Git и фиксировать изменения
- ③ Изучили принцип снимков (коммитов) Git при работе с файлами



Работа с историей



Коммиты и история проектов в Git



Какие возможности даёт коммит в Git

- 1 Отслеживать, кто, что и когда когда изменял
- 2 Возвращаться к предыдущему состоянию файла
- 3 Переписывать историю файлов



Скринкаст

Внесение изменений в проект



Внесение изменений в файл Git

- 1 В Visual Studio Code откройте файл **README.md** в папке **test**
- 2 Внесите изменения в файл, добавив текст «Я учусь в Нетологии». Сохраните изменения
- 3 Проверьте статус файла через команду **git status**. Файл **README.md** подсвечен красным и рядом с ним написано **modified**. Значит, что после последнего сохранения проекта файл изменился

```
mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ |
```

Вид из терминала Windows

Внесение изменений в файл Git

- 4 Добавьте файл в индекс и подпишите коммит, выполнив последовательно 2 команды **git add README.md** и **git commit -m "change README"**
- 5 Вновь проверьте статус файла через команду **git status**. Git выдаст сообщение, что ничего в проекте не изменилось с последнего сохранения

```
o o o
mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git add README.md

mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git commit -m "change README"
[main 2199e0e] change README
1 file changed, 3 insertions(+), 1 deletion(-)

mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git status
On branch main
nothing to commit, working tree clean
```

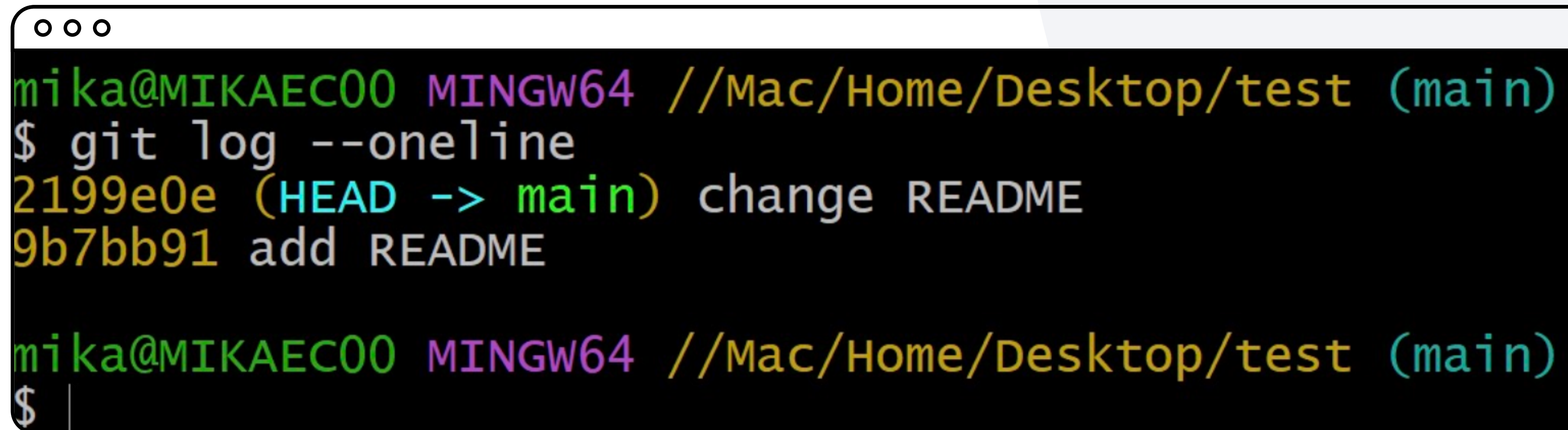

Скринкаст

Просмотр истории проекта



Просмотр истории проекта

- 1 Выполните команду **git log --oneline**

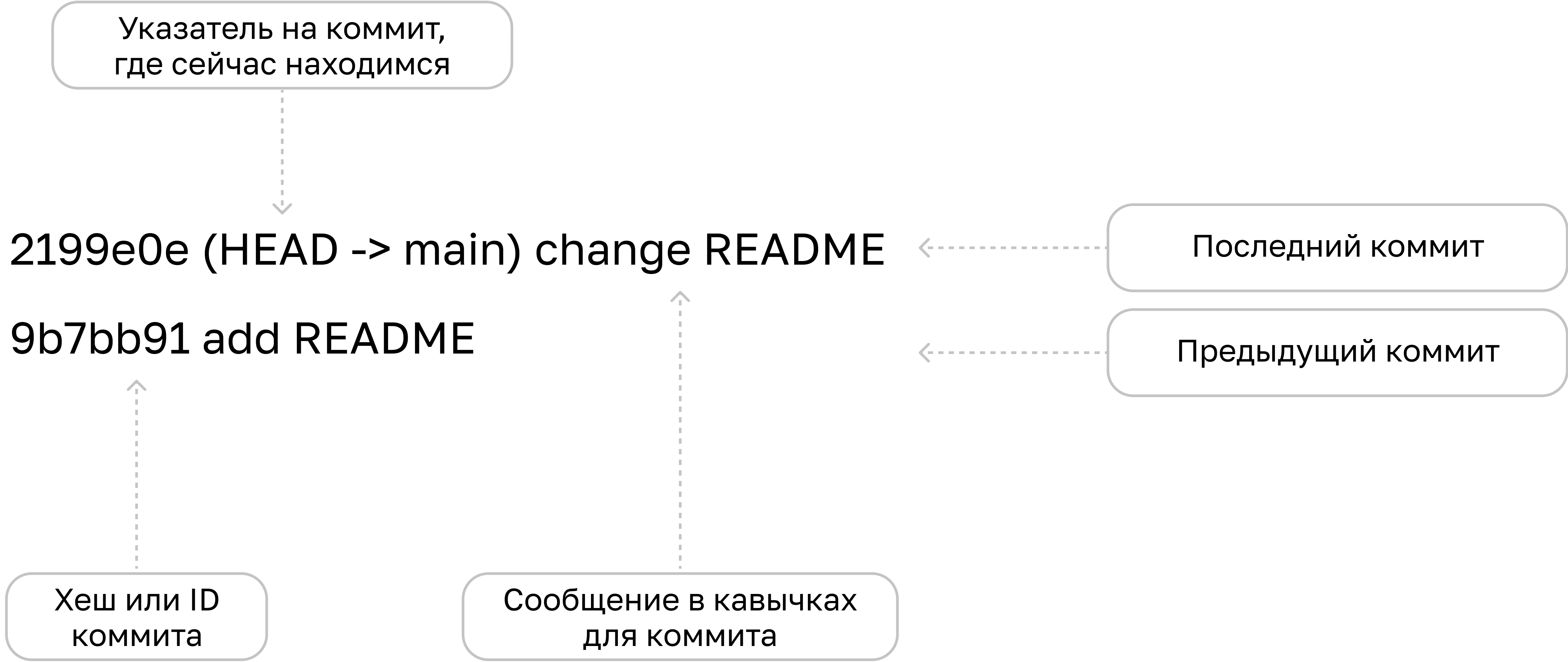


A terminal window with a black background and white text. The window title bar shows three small circles. The prompt is 'mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)'. The command '\$ git log --oneline' has been executed, resulting in two lines of output: '2199e0e (HEAD -> main) change README' and '9b7bb91 add README'. The prompt '\$' is shown again with a vertical cursor line.

```
mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git log --oneline
2199e0e (HEAD -> main) change README
9b7bb91 add README

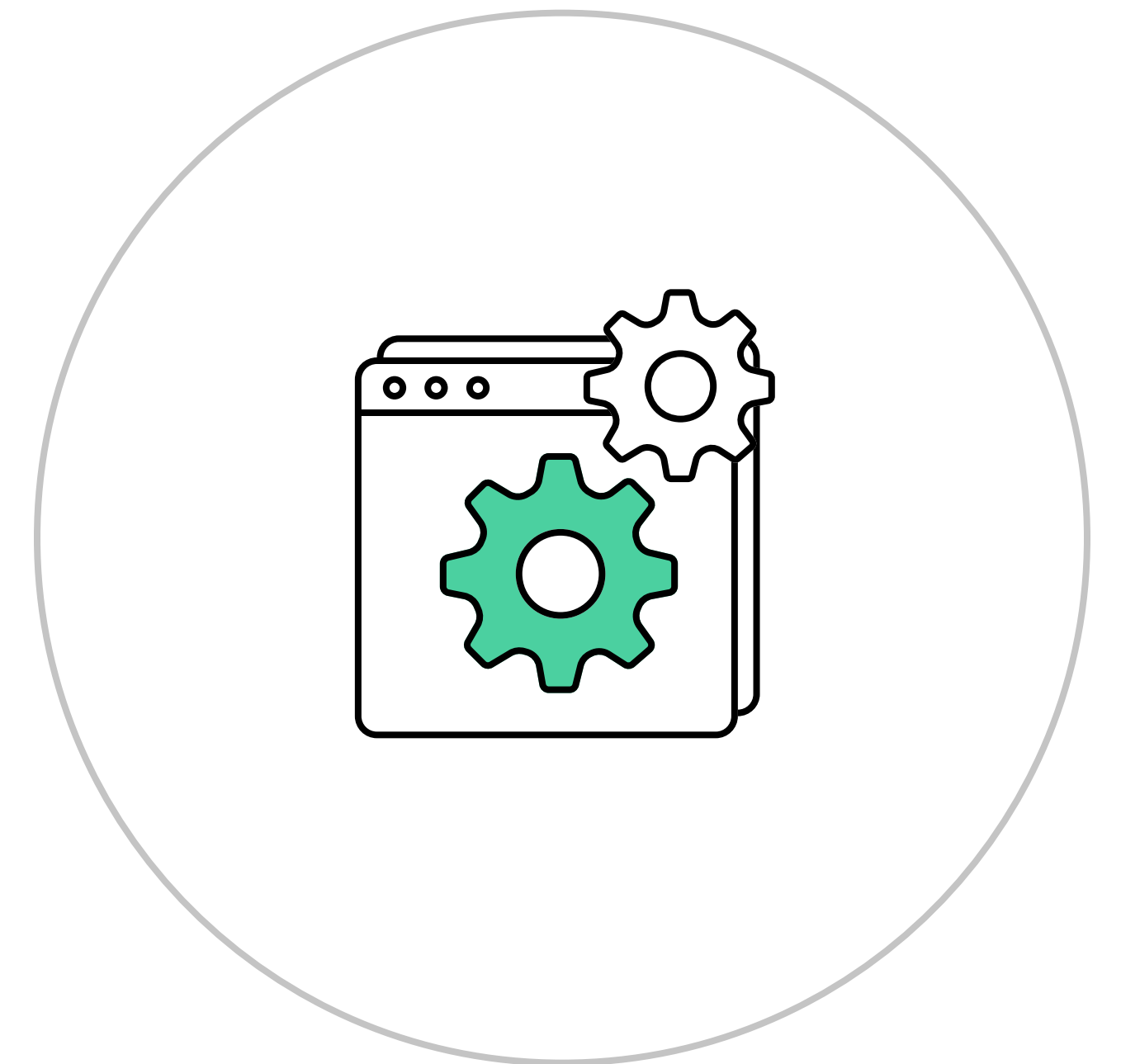
mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ |
```

Просмотр истории проекта



Флаг

Способ указать для команды дополнительные параметры.
Как правило, в Git флаг начинается с одного (-)
или двух (--) дефисов



Просмотр истории проекта

- 1 Чтобы посмотреть всю информацию о коммите, выполните команду **git log** без флага. В такой записи мы можем увидеть полный хеш коммита, автора и дату с точным временем.
Если история очень длинная, то Git может перейти в режим вывода данных и не давать вводить новые команды. В этом случае нажмите **Q** — режим вывода закроется и вы сможете продолжить работу

```
ooo
mika@MIKAE00 MINGW64 //Mac/Home/Desktop/test (main)
$ git log
commit 2199e0e8cac0cff96c17dbf4dcc2fc3e3451cdc6 (HEAD -> main)
Author: Emma Paris <eparis@atlassian.com>
Date: Thu Jun 9 22:48:17 2022 +0400

    change README

commit 9b7bb9156a8a2adaefbac549a2900d3b2f661dad
Author: Emma Paris <eparis@atlassian.com>
Date: Thu Jun 9 22:43:34 2022 +0400

    add README
```

Вид из терминала Windows

Скринкаст

Исправление коммита



Исправление коммита

- 1 Создайте коммит с ошибкой в подписи. Для этого напишите что-нибудь в файле **README.md**, сохраните файл и **закоммитьте его с подписью "ewrror"**. Выведите историю с помощью **git log**:

```
c0f448b (HEAD -> main) ewrror  
f8bb0de change README  
bb02449 add README
```

- 2 Исправьте подпись последнего коммита. Выполните команду **git commit --amend -m "error"** и вновь выведите историю с помощью **git log**:

```
262abc8 (HEAD -> main) error  
f8bb0de change README  
bb02449 add README
```

- 3 Файл перезаписался. У коммита изменился хеш, так как Git отменил прошлый коммит и создал новый, исправленный

Исправление коммита

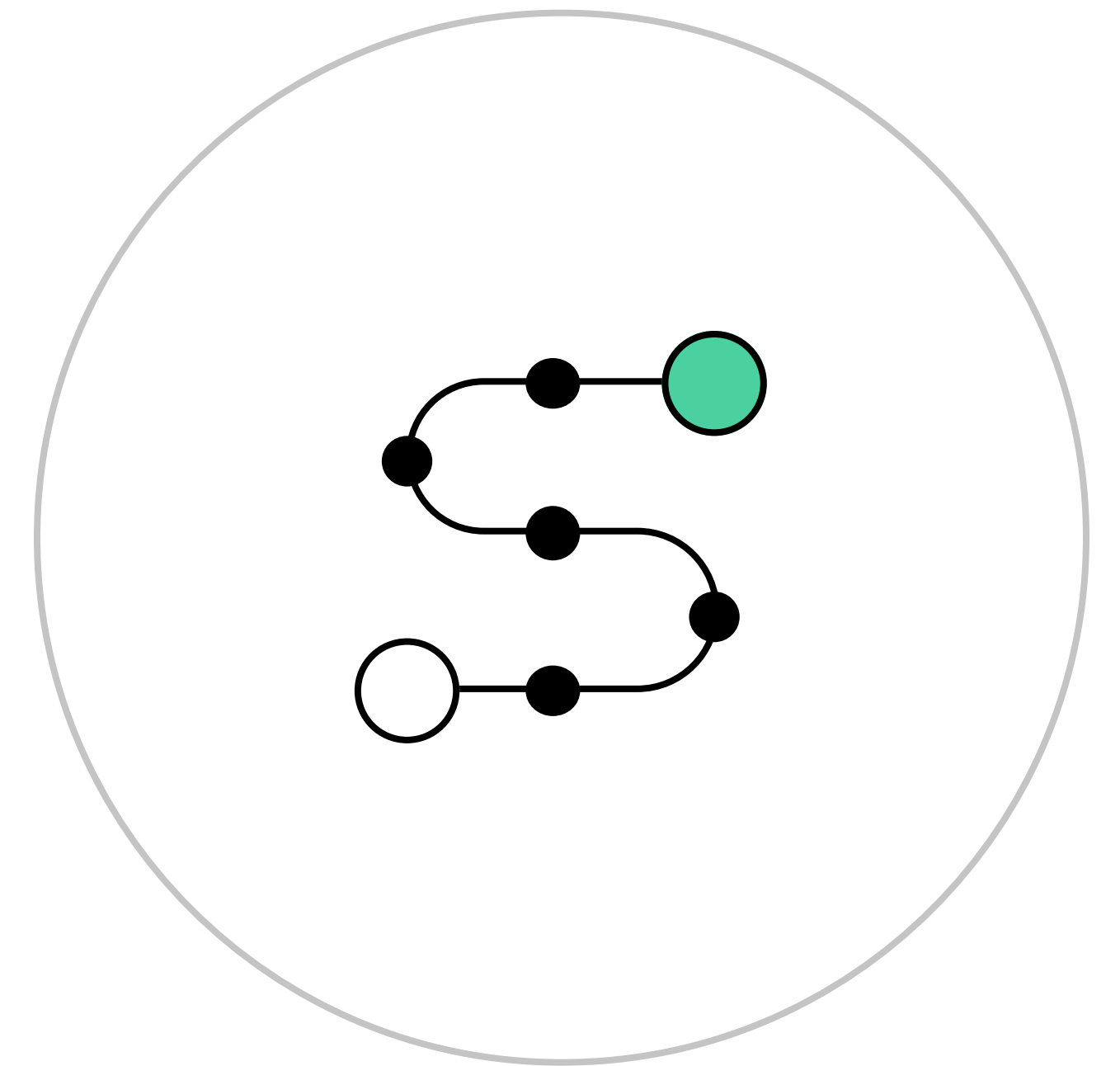
- 4 Создайте в папке test ещё один файл с названием **errors.md** и внутри напишете:

```
# Ошибки, которые нужно исправить
```

- 5 Сохраните файл errors.md и добавьте его в отслеживаемые через команду **git add имя файла**
- 6 Можно добавить в отслеживаемые сразу много файлов. Тогда нужно воспользоваться сокращением и написать **git add -A**. В коммит попадут все файлы из текущей папки, которые изменились с последнего коммита
- 7 Поместите файл errors.md в последний коммит — снова используем команду **git commit --amend -m "error"**. Теперь в последнем коммите хранится не только последняя версия файла README.md, но и новый файл errors.md

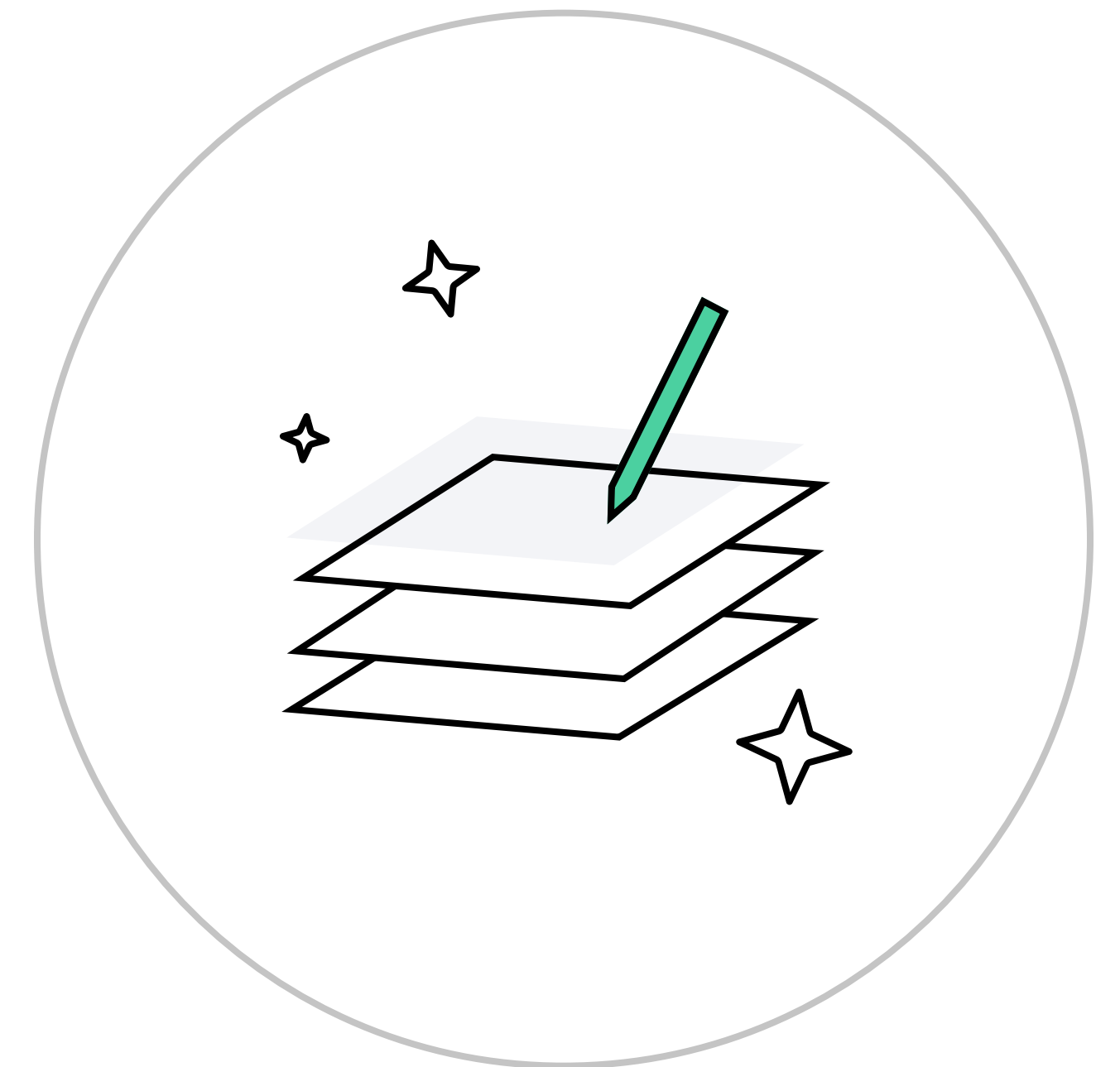
Суть исправления коммита

Когда вносите правки в последний коммит, то не столько его исправляете, сколько заменяете новым, который полностью его перезаписывает. В результате всё выглядит так, будто первоначальный коммит никогда не существовал, он больше не появится в истории репозитория



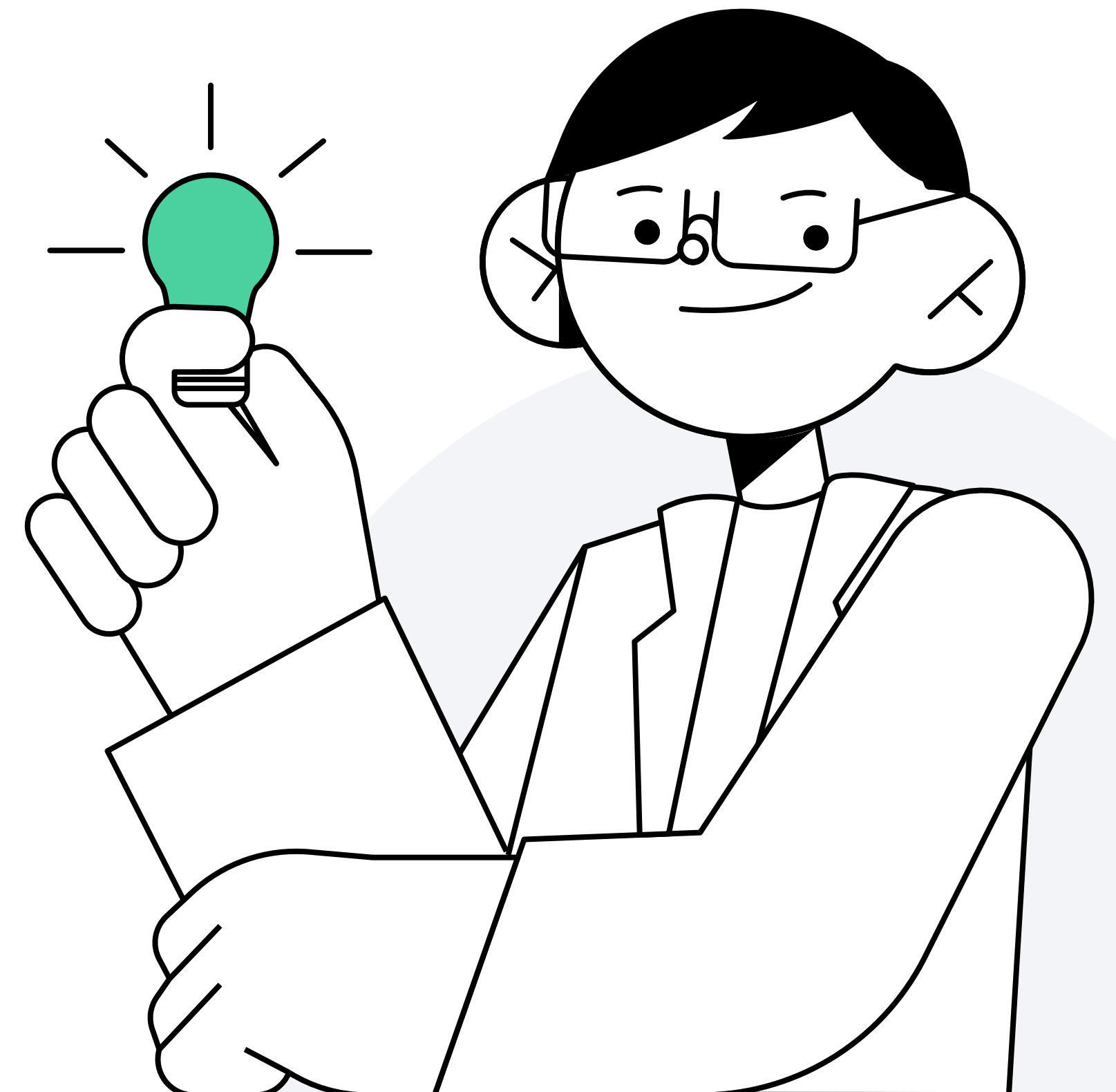
Использование исправления коммита

Смысл изменения коммитов в добавлении незначительных правок в последние коммиты и в избежании засорения истории сообщениями типа «Исправление грамматической ошибки»



Итоги

- 1 Научились вносить изменения в проект и фиксировать данные обновления в Git
- 2 Научились вызывать историю изменений проекта и правильно читать данные
- 3 Научились исправлять коммит в проекте



Путешествие по истории проекта

Скринкаст

Переключение между коммитами проекта



Перемещение по истории проекта

- 1 Выведите историю проекта через команду **git log**
- 2 Найдите **хеш прошлого коммита**, к которому хотите вернуться. Достаточно первых 7 символов
- 3 Переключитесь на нужный коммит с помощью команды **git checkout хеш коммита**

Перемещение по истории проекта

4 Git выдаст сообщение:

Note: switching to 'bb02449'.

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you make in this state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may do so (now or later) by using -c with the switch command. Example:

```
git switch -c <new-branch-name>
```

Or undo this operation with:

```
git switch -
```

Turn off this advice by setting config variable advice.detachedHead to false
HEAD is now at bb02449 add README

Перемещение по истории проекта

- 5 Откройте **Visual Studio Code** и увидите, что в файле README.md написан только заголовок `# Hello, world!`, а файла errors.md больше нет в папке. Проект вернулся на точку сохранения, когда нового файла ещё нет, а в README был только заголовок
- 6 Если проверить историю проекта, то увидим только один этот коммит. Коммиты, сделанные позже, отображаться не будут:

```
$ git log --oneline  
bb02449 (HEAD) add README
```

- 7 Если вызвать команду **git log с флагом --all**, то Git покажет всю историю проекта, вне зависимости от того, на каком коммите сейчас находитесь (см. указатель HEAD):

```
$ git log --all --oneline  
ef0d05c (main) error  
f8bb0de change README  
bb02449 (HEAD) add README
```

Перемещение по истории проекта

- 8 Второй вариант — использовать вместо конкретного хеша коммита **тире** или **git checkout main**:

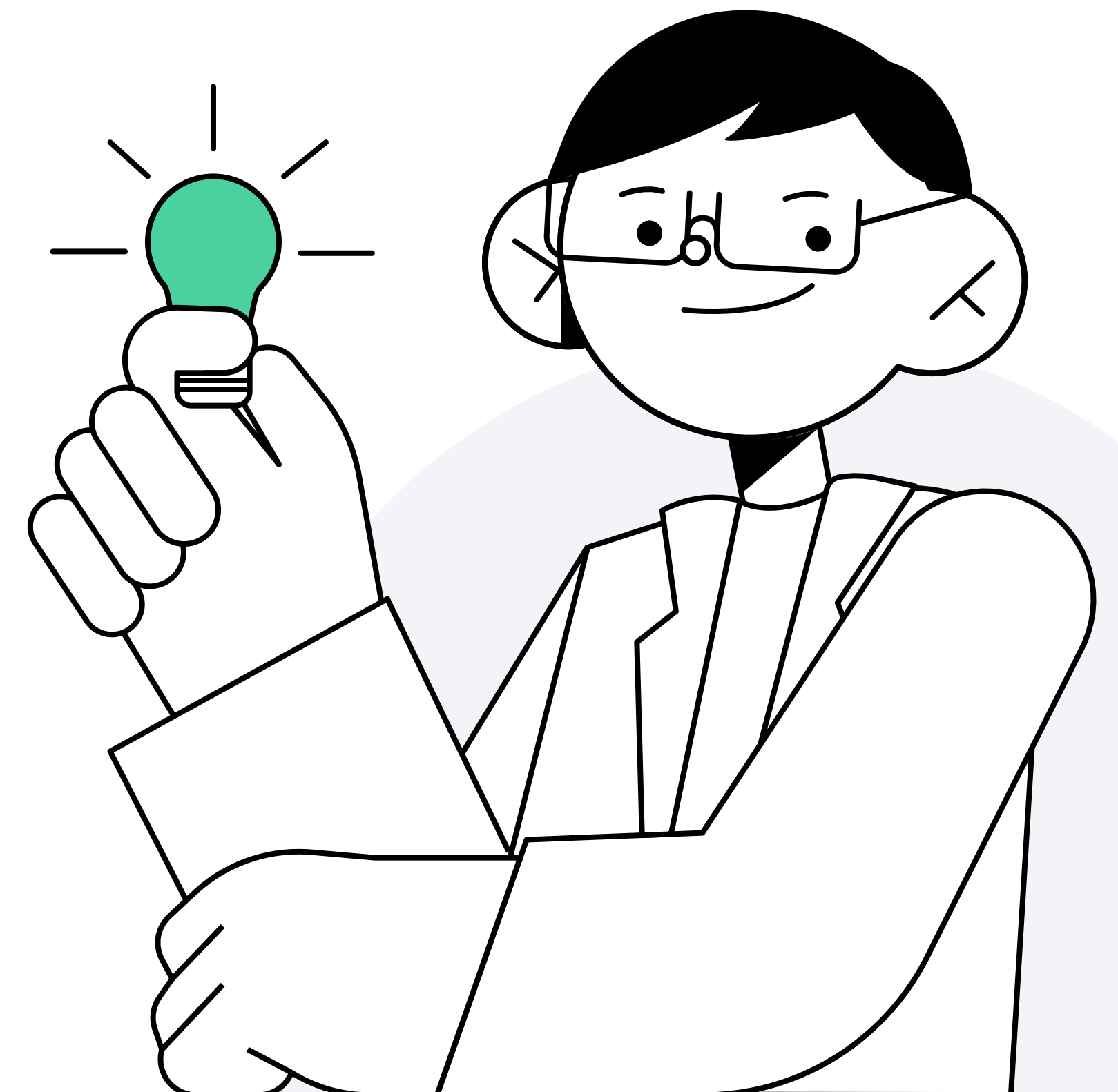
```
$ git checkout -  
Previous HEAD position was bb02449 add README  
Switched to branch 'main'
```

- 9 Проверьте, что вернулись к последней версии проекта через команду **git log --oneline**:

```
$ git log --oneline  
ef0d05c (HEAD -> main) error  
f8bb0de change README  
bb02449 README added
```

Итоги

- 1 Научились перемещаться по истории проекта в Git, используя хеши коммитов



Резюме

- Смотреть историю можно командой **git log** с разными флагами **--oneline** или **--all**
- Исправить что-то в последнем коммите, который хранится только на компьютере, можно при помощи флага **--amend**
- Чтобы не добавлять в коммит все файлы по именам можно, написать **флаг -A**
- Переключиться между коммитами проекта можно командой **git checkout хеш**.
Вернуться к последнему коммиту можно командой **git checkout -**



Спасибо за внимание!

Алёна Батицкая
Фронтенд-разработчик, фрилансер

